COMPLETE STUDY NOTES • BEGINNER → ADVANCED

DATABASE MANAGEMENT SYSTEMS



DBMS • SQL • Normalization • Transactions • NoSQL • Data Warehousing

For Computer Science, Data Science, IT and Software Engineering students

01 Introduction to DBMS

02 Database Models & Design

03 Database Normalization

04 Structured Query Language (SQL)

05 Relational Algebra & Calculus

06 Transaction Management

07 Storage & Optimization

08 Security & Recovery

09 Advanced Database Systems

10 Practical Database Development

10 PARTS • 28 TOPICS • DIAGRAMS • WORKED EXAMPLES • EXAM TIPS

Colour-coded tables, definitions, SQL code blocks and quick-revision sheets in every part.

Table of Contents

Part 1: Introduction to Database Management Systems 4

Topic 1: Fundamentals of Database Systems 4

Topic 2: Database Architecture 7

Part 2: Database Models and Design 10

Topic 3: Database Models 10

Topic 4: Entity Relationship Diagrams (ERD) 11

Topic 5: Relational Database Model 15

Part 3: Database Normalization 17

Topic 6: Functional Dependencies 17

Topic 7: Database Normalization 19

Part 4: Structured Query Language (SQL) 24

Topic 8: Introduction to SQL 24

Topic 9: SQL Command Categories 26

Topic 10: SQL Practical Operations 26

Topic 11: SQL Joins 28

Topic 12: Advanced SQL 30

Part 5: Relational Algebra and Calculus 33

Topic 13: Relational Algebra 33

Topic 14: Relational Calculus 34

Part 6: Transaction Management 36

Topic 15: Database Transactions 36

Topic 16: Concurrency Control 38

Part 7: Database Storage and Optimization 41

Topic 17: Database Storage Management 41

Topic 18: Indexing and Hashing 42

Topic 19: Query Processing and Optimization 44

Part 8: Database Security and Recovery 46

Topic 20: Database Security 46

Topic 21: Backup and Recovery 48

Part 9: Advanced Database Systems 50

Topic 22: Distributed Databases 50

Topic 23: NoSQL Databases 52

Topic 24: Data Warehousing and OLAP 53

Topic 25: Modern Database Technologies 55

Part 10: Practical Database Development 57

Topic 26: MySQL Practical Training 57

Topic 27: Connecting Databases to Programming Languages 58

Topic 28: Real-World Database Projects 59

Recommended Study Order and Roadmap 61

Quick Revision Sheet 62

How to Use These Notes

Symbol	What it means
DEFINITION (blue box)	Formal meaning of a term - learn these word for word.
KEY POINT (green box)	A rule or fact that is almost always examined.
EXAMPLE (orange box)	Worked example that shows the idea in practice.
WATCH OUT (red box)	Common mistakes, dangers and exceptions.
EXAM TIP (purple box)	Memory aids and how to answer exam questions.
GOOD TO KNOW (teal box)	Extra context and practical advice.
Dark code block	SQL / Python code you should type and run yourself.

EXAM TIP

Study in the order of the parts (Part 1 → Part 10). Read a topic, close the notes, and write its key points from memory. Then run every SQL example on a real database such as MySQL or SQLite.

PART 1 OF 10

Introduction to Database Management Systems

1

The foundations: what data, information, databases and DBMS are, why they matter, and how a database system is

IN THIS PART

Topic 1 Fundamentals of Database Systems
Topic 2 Database Architecture

TOPIC 1

Fundamentals of Database Systems

1.1 Data, Information, Database and DBMS

Term	Meaning	Example
Data	Raw, unprocessed facts and figures with no context or meaning on their own.	25, "Mary", 5000, 2026-10-14
Information	Data that has been processed, organised and given meaning so it supports decisions.	"Mary is 25 years old and earns K5,000 per month."
Database	An organised, shared collection of logically related data, stored electronically and describing an organisation's activities.	A university database holding students, courses, results.
DBMS	Software that lets users define, create, store, retrieve, update, secure and share a database.	MySQL, PostgreSQL, Oracle, SQL Server, MongoDB.
Database System	Database + DBMS + application programs + hardware + people.	The whole bus-booking system of a transport company.

KEY POINT
Data → **processing** → Information → **decision**. A database stores the data; a DBMS is the manager that controls access to it.

1.2 History and Evolution of Databases

Era	Technology	What changed
1950s-60s	File-based / punch cards / magnetic tape	Sequential processing; each program managed its own files.
1960s	Hierarchical (IMS) and Network (CODASYL) models	First true DBMS; navigational access with pointers.
1970	Relational model - E. F. Codd (IBM)	Data in tables; mathematical foundation; data independence.
1970s-80s	SQL, System R, Oracle, DB2, Ingres	SQL standardised (ANSI 1986, ISO 1987); commercial RDBMS.
1976	Entity-Relationship model - Peter Chen	Standard way to design databases conceptually.
1990s	Object-oriented and Object-relational DBMS; data warehouses	Complex data types; OLAP and decision support.
2000s	XML, web databases, open source (MySQL, PostgreSQL), NoSQL	Web-scale data; flexible schemas; horizontal scaling.
2010s-now	Cloud DBaaS, NewSQL, in-memory, graph, time-series, vector databases	Big data, AI, real-time analytics, global scale.

1.3 Characteristics of a DBMS

- **Self-describing nature** - stores the catalog (metadata) with the data
- **Insulation** between programs and data (data independence)
- **Data abstraction** - hides storage details from users
- **Multiple views** of the same data for different users
- **Sharing** and multi-user transaction processing
- **Controlled redundancy** - duplicate data kept to a minimum
- **Integrity constraints** - rules that keep data valid
- **Security and access control** - authentication and privileges
- **Backup and recovery** from crashes and failures
- **Persistent storage** of large amounts of data
- **Efficient query processing** with indexes and optimisers
- **ACID transactions** for reliable updates

1.4 Objectives and Importance of a DBMS

Objective	Why it matters
Reduce data redundancy and inconsistency	One fact stored once - no conflicting copies of a student's address.
Provide data independence	Change storage or structure without rewriting every program.
Ensure data integrity and accuracy	Constraints reject invalid data (negative age, missing key).
Enable data sharing	Many users and applications use the same up-to-date data at once.
Protect data (security)	Only authorised people see or change sensitive data.
Provide backup and recovery	Data survives power loss, crashes and human error.
Support fast, flexible querying	Answer new questions with SQL without writing new programs.
Support concurrent access	Hundreds of bookings or payments processed safely at the same time.

1.5 Applications of Databases in Real Life

Sector	Typical database use
Banking & Mobile Money	Accounts, balances, transactions, loans, customer KYC records.
Education	Student records, admissions, timetables, examinations, library.
Healthcare	Patient records, prescriptions, appointments, lab results, billing.
Transport & Travel	Bus, train and airline ticket booking, schedules, seat availability.
Retail & E-commerce	Products, inventory, orders, payments, recommendations.
Telecommunications	Subscribers, call records, billing, network data.
Government	National registration, tax, land records, elections, licences.
Social media & Web	Profiles, posts, friendships (graph), messages, analytics.
Science & Research	Genomic data, sensor and bird-migration tracking, climate records.

1.6 Advantages and Disadvantages of a DBMS

Advantages	Disadvantages
Reduced redundancy and inconsistency	High initial cost of software, hardware and training
Data sharing among many users and applications	Complexity - needs skilled designers and DBAs
Improved data integrity through constraints	Performance overhead for simple, small tasks
Better security and access control	Single point of failure if no backup / replication
Backup and recovery services	Conversion cost from existing file systems
Data independence and easier maintenance	Higher impact when a failure or attack occurs
Standardised access (SQL) and faster development	Needs ongoing administration and tuning

1.7 Traditional File Processing vs DBMS

DEFINITION

In a **file-processing system** each application keeps its own separate files (e.g. text, spreadsheets). Data and programs are tightly bound together, which causes duplication and inconsistency.

Feature	File System	DBMS
Redundancy	High - same data repeated in many files	Controlled - data stored once and shared
Consistency	Copies get out of sync (inconsistent)	Single source of truth; constraints enforced
Data independence	None - programs depend on file format	Physical and logical independence
Sharing / concurrency	Poor; file locking problems	Excellent multi-user control
Security	Basic OS-level permissions only	Fine-grained: users, roles, privileges
Querying	New program needed for each new question	Ad hoc queries using SQL
Backup & recovery	Manual and unreliable	Automatic logs, checkpoints, recovery
Integrity	Enforced by each program (if at all)	Enforced centrally by the DBMS
Cost / complexity	Cheap and simple for small tasks	More costly and complex but scalable

1.8 Database Users and Their Responsibilities

User type	Responsibility
Database Administrator (DBA)	Installs, configures, secures, tunes and backs up the DBMS; manages users and storage.
Database Designer	Identifies data to store, chooses structures, builds ER diagrams and schemas.
Application Programmer / Developer	Writes programs (Python, Java, PHP) that connect to and use the database.
Casual (Naive) End User	Uses ready-made forms and apps (e.g. a bank customer using an ATM).
Sophisticated End User	Writes their own queries (analysts, engineers, data scientists).
System Analyst	Studies requirements and specifies transactions for end users.
Data Analyst / Scientist	Queries and mines the data for insights and reports.

1.9 Roles of a Database Administrator (DBA)

- Installing and upgrading the DBMS
- Defining schemas and storage structures
- Creating user accounts, roles and privileges
- Enforcing security policies and auditing
- Scheduling backups and testing recovery
- Monitoring performance and tuning queries / indexes
- Capacity planning and storage management
- Ensuring high availability and disaster recovery
- Applying patches and migrating data
- Documenting and training users

1.10 Examples of Database Management Software

Software	Type	Typical use
MySQL / MariaDB	Open-source relational	Web applications, PHP/Python stacks
PostgreSQL	Open-source object-relational	Advanced SQL, GIS (PostGIS), analytics
Oracle Database	Commercial relational	Large enterprises, banking, telecom
Microsoft SQL Server	Commercial relational	Windows / .NET business systems
SQLite	Embedded relational (file-based)	Mobile apps, browsers, small projects
IBM Db2	Commercial relational	Mainframes, finance
MongoDB	NoSQL document	Flexible JSON data, web apps
Redis	NoSQL key-value / in-memory	Caching, sessions, real-time counters
Cassandra	NoSQL column-family	Massive write-heavy distributed data
Neo4j	NoSQL graph	Social networks, recommendations, fraud

TOPIC 2

Database Architecture

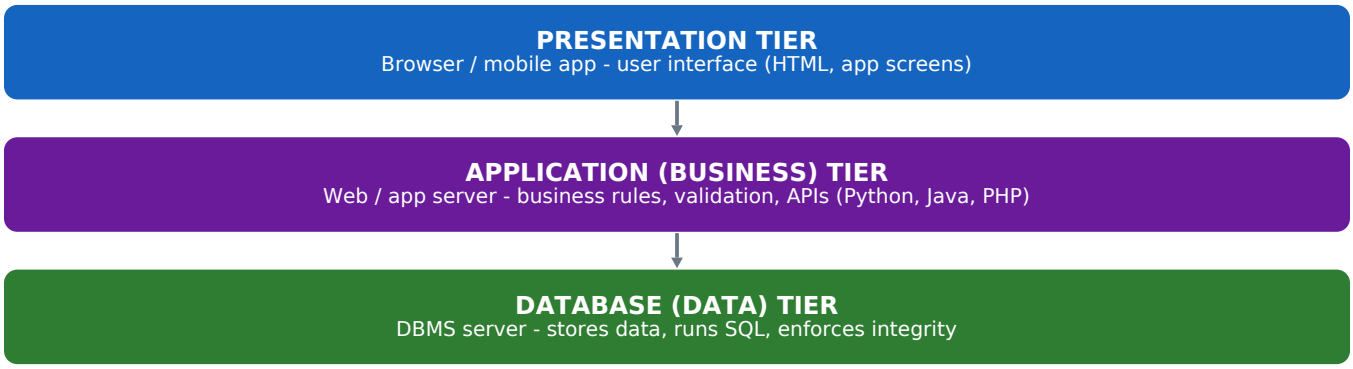
2.1 Introduction to Database Architecture

DEFINITION

Database architecture describes how the components of a database system - users, application programs, DBMS engine and stored data - are arranged and communicate.

2.2 One-, Two- and Three-Tier Architectures

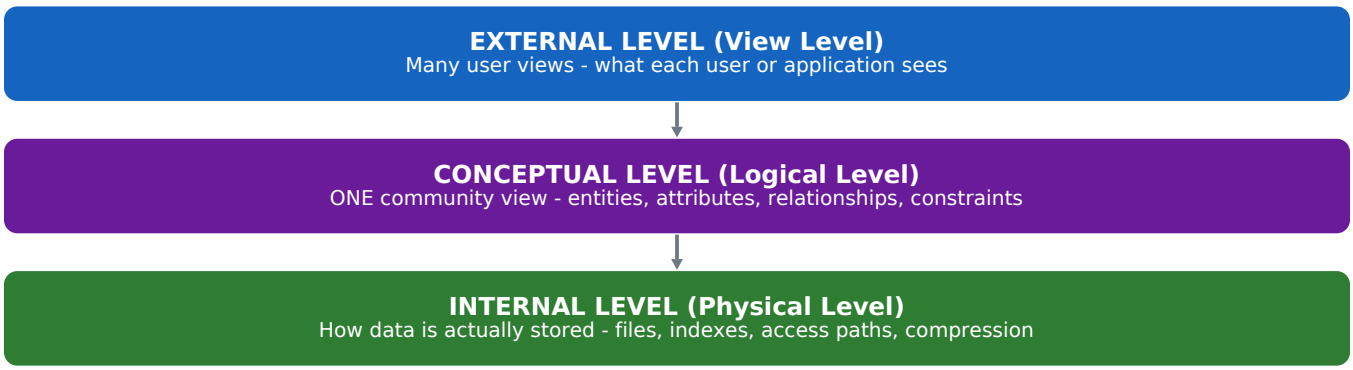
Architecture	How it works	Pros	Cons	Example
1-Tier (Single-tier)	User, application and database all on the **same machine**; the user works directly with the DBMS.	Simple, no network needed, cheap	Single user, no sharing, weak security	MS Access or SQLite on a laptop
2-Tier (Client-Server)	Client application talks **directly** to the database server (via ODBC/JDBC).	Faster, simple to build	Business logic on clients; hard to scale & secure	Desktop app connected to a SQL Server
3-Tier	Client (presentation) → **Application server** (business logic) → Database server (data).	Scalable, secure, easy to maintain; DB hidden from users	More complex and costly to build	Web apps, mobile money, online shopping



2.3 ANSI/SPARC Three-Schema Architecture

DEFINITION

Proposed by ANSI/SPARC, the **three-schema architecture** separates the database into three levels so users are insulated from physical storage details. Its goal is **data independence**.



Level	Describes	Who uses it	Example
External	Part of the database relevant to a user group (views, forms, reports).	End users, application programmers	A teacher sees only grades of own class; a bursar sees fees.
Conceptual	The whole logical structure for the entire community: tables, keys, constraints, relationships.	DBA, database designers	STUDENT(ID, Name, DOB), COURSE(Code, Title), ENROLMENT(...)
Internal	Physical storage: file organisation, indexes, record formats, data compression, encryption.	DBA, system programmers	B+ tree index on StudentID; heap file on disk.

2.4 Data Abstraction and Data Independence

DEFINITION

Data abstraction hides the complexity of storage from users by presenting only the level of detail they need (physical → logical → view).

Independence	Meaning	Which mapping absorbs the change?
Physical data independence	Ability to change the **internal schema** (storage devices, file organisation, indexes) without changing the conceptual schema or applications.	Conceptual / Internal mapping
Logical data independence	Ability to change the **conceptual schema** (add a column or table) without changing external views or applications.	External / Conceptual mapping (harder to achieve)

2.5 Database Schemas and Instances

Concept	Definition	Changes how often?	Analogy
Schema (intension)	The overall **design / structure** of the database: tables, columns, types, constraints.	Rarely	Blueprint of a building
Instance (extension)	The **actual data** stored in the database at a particular moment (snapshot / state).	Constantly	The people inside the building today

EXAMPLE

Schema: **STUDENT(StudentID INT, Name VARCHAR(50), Age INT)** | Instance: (101, "Mary", 20), (102, "John", 22)

2.6 Database Languages and Interfaces

Language	Purpose	Commands
DDL - Data Definition	Define and change the structure (schema) of the database	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML - Data Manipulation	Insert, modify and delete the data (procedural or declarative)	INSERT, UPDATE, DELETE
DQL - Data Query	Retrieve data	SELECT
DCL - Data Control	Control access and privileges	GRANT, REVOKE
TCL - Transaction Control	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

Interface	Description
Menu-based / Forms-based	Pick options or fill in forms - for casual and naive users.
Graphical User Interface (GUI)	Tools like MySQL Workbench or phpMyAdmin with diagrams and query builders.
Command-line / Query language	Type SQL directly in a terminal (mysql, psql, sqlite3).
Natural-language interface	Ask questions in plain English; the system translates to SQL (increasingly AI-based).
Programmatic interface	APIs: JDBC, ODBC, Python DB-API, REST - used by application programmers.
Web interface / Admin interface	Browser-based admin consoles; DBA tools for backup, users, monitoring.

PART 2 OF 10

Database Models and Design

2

How data is modelled: classic database models, Entity-Relationship diagrams, the relational model and keys.

IN THIS PART

Topic 3 Database Models
Topic 4 Entity Relationship Diagrams (ERD)
Topic 5 Relational Database Model

TOPIC 3

Database Models

DEFINITION
A **data model** is a collection of concepts and rules for describing the structure of a database: its data, relationships, constraints and (often) operations.

3.1 The Main Database Models

Model	Structure	Key idea	Example systems
Hierarchical	Tree: each child has **one parent**	Parent-child (1:N) links via pointers; fast navigation from root	IBM IMS, Windows Registry, XML
Network	Graph: a child may have **many parents**	Records linked by sets/pointers; supports M:N	CODASYL, IDMS, Integrated Data Store
Relational	Tables (relations) of rows and columns	Data linked by key values, not pointers; based on set theory	MySQL, PostgreSQL, Oracle, SQL Server
Object-Oriented	Objects with attributes + methods	Stores objects directly; supports inheritance & encapsulation	ObjectDB, db4o, Versant
Object-Relational	Tables extended with objects / UDTs	Relational + user-defined types, inheritance, arrays	PostgreSQL, Oracle, Informix
Entity-Relationship	Entities, attributes, relationships	Conceptual (design) model, drawn as diagrams	Design tool - mapped to relational
NoSQL	Key-value, document, column, graph	Flexible schema, horizontal scaling, BASE	MongoDB, Redis, Cassandra, Neo4j

3.2 Advantages and Disadvantages of Each Model

Model	Advantages	Disadvantages
Hierarchical	Simple; very fast for parent-child lookups; data integrity along tree	Only 1:N; no M:N; rigid; hard to restructure; redundancy
Network	Supports M:N; flexible access paths; fast	Complex pointer structure; hard to design and maintain; navigational programming
Relational	Simple tables; data independence; powerful SQL; strong theory; mature	Can be slow for huge joins; poor for complex/unstructured data; rigid schema
Object-Oriented	Handles complex data (CAD, multimedia); reuse through inheritance	Less standard; steeper learning curve; fewer tools; weak query support
Object-Relational	Best of both - SQL plus complex types; backward compatible	More complex than pure relational; vendor-specific features
Entity-Relationship	Easy to understand; excellent for communication with users	Only conceptual - must be converted; no standard notation
NoSQL	Scales out; schema-flexible; fast for specific workloads	Weaker consistency (eventual); no standard query language; limited joins

3.3 Comparison Summary

Feature	Hierarchical	Network	Relational	Object-Oriented	NoSQL
Structure	Tree	Graph	Tables	Objects	Varies
Relationships	1:N only	1:N, M:N	All via keys	References	Embedded / links
Query language	DL/I (navigational)	DML navigation	SQL	OQL	Own APIs / query langs
Flexibility	Low	Medium	High	High	Very high
Data independence	Low	Low	High	Medium	High
Use today	Legacy / XML	Legacy	Dominant	Niche	Big data & web

EXAM TIP

Remember: **Hierarchical** = tree (one parent); **Network** = graph (many parents); **Relational** = tables + keys. The relational model replaced the others because it separates logical design from physical storage and uses declarative SQL.

TOPIC 4

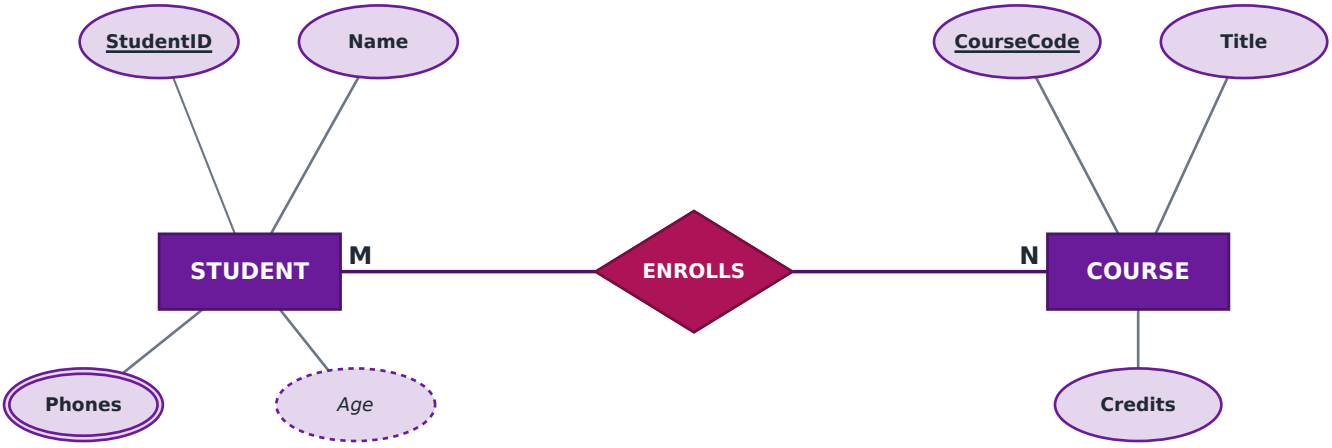
Entity Relationship Diagrams (ERD)

4.1 Introduction to ER Diagrams

DEFINITION

An **Entity-Relationship Diagram** (Peter Chen, 1976) is a visual blueprint of a database showing the **entities** (things), their **attributes** (properties) and the **relationships** between them. It is drawn *before* any tables are created.





Many-to-many (M:N): a student takes many courses; a course has many students

4.2 Entities and Entity Sets

Term	Meaning	Example
Entity	A real-world object or concept that can be identified uniquely and about which we store data.	A student named Mary; bus UAA 123
Entity set (type)	A collection of similar entities sharing the same attributes.	STUDENT, BUS, COURSE
Entity instance	One specific member of the entity set.	Student 101

4.3 Attributes and Their Types

Attribute type	Description	Example	Symbol
Simple (atomic)	Cannot be divided further	Age, Gender	Single ellipse
Composite	Made of sub-parts	Name → FirstName + LastName; Address → Street, City, Province	Ellipse with child ellipses
Single-valued	Exactly one value per entity	Date of birth, NRC number	Single ellipse
Multivalued	Can hold several values	Phone numbers, Skills, Email addresses	Double ellipse
Derived	Calculated from other attributes; not stored	Age (from DOB), Total price	Dashed ellipse
Key attribute	Uniquely identifies each entity	StudentID, NRC, ISBN	Underlined text
Null-able / optional	May be empty if unknown or not applicable	Middle name, Fax number	-

4.4 Relationships, Degree, Cardinality and Participation

Concept	Explanation
Relationship	An association among entities, usually named with a verb: Student *enrolls in* Course.
Degree	Number of entity sets in a relationship: **Unary** (1, recursive: Employee manages Employee), **Binary** (2, most common), **Ternary** (3: Doctor-Patient-Drug), n-ary.
Cardinality ratio	Maximum number of instances that can be associated: 1:1, 1:N, N:1, M:N.
Participation (existence) constraint	**Total** (mandatory, double line): every entity must take part. **Partial** (optional, single line): some may not.
Min-max notation	(min, max) such as (0,N) or (1,1) shows exact participation and cardinality.

Cardinality	Meaning	Example	Implement as
One-to-One (1:1)	One A relates to at most one B, and vice versa	Person - Passport; Country - Capital	FK (unique) in either table

Cardinality	Meaning	Example	Implement as
One-to-Many (1:N)	One A relates to many B; each B has one A	Department - Employees; Mother - Children	FK on the "many" side
Many-to-Many (M:N)	Many A relate to many B	Student - Course; Passenger - Bus trip	New junction (bridge) table

4.5 Strong and Weak Entities

	Strong entity	Weak entity
Key	Has its own primary key	No full key; has only a **partial key** (discriminator)
Existence	Exists independently	Depends on an owner (identifying) entity
Symbol	Single rectangle	Double rectangle
Relationship	Normal diamond	Identifying relationship - double diamond, total participation
Primary key in tables	Own attribute(s)	Owner's PK + partial key (composite)
Example	EMPLOYEE (EmpID)	DEPENDENT (Name) of EMPLOYEE; Seat of a Bus; Order line of an Order

4.6 Enhanced ER (EER) Model

DEFINITION
EER adds object-oriented ideas to the ER model: **subclasses/superclasses**, **specialization**, **generalization**, **inheritance** and **aggregation**.

Concept	Direction	Explanation	Example
Specialization	Top-down	Split a general entity into more specific sub-entities with extra attributes.	EMPLOYEE → Lecturer, Technician, Administrator
Generalization	Bottom-up	Combine similar entities into a common super-entity.	Car, Bus, Truck → VEHICLE
Inheritance	-	Sub-entities inherit attributes and relationships of the super-entity (IS-A).	Lecturer inherits Name, ID from EMPLOYEE
Constraints on specialization	-	**Disjoint (d)** vs **Overlapping (o)**; **Total** vs **Partial** completeness.	A person can be both Student and Staff = overlapping
Aggregation	-	Treat a relationship (with its entities) as a single higher-level entity so it can take part in another relationship.	(Employee works on Project) is sponsored by Client

4.7 Converting ER Diagrams into Relational Tables

ER element	Mapping rule
Strong entity	Create a table; each simple attribute → a column; key attribute → PRIMARY KEY.
Composite attribute	Store only its simple components as separate columns (FirstName, LastName).
Multivalued attribute	Create a **new table**: (OwnerPK, value) with a composite PK; FK back to the owner.
Derived attribute	Do not store - compute using a query / view (or store only for performance).
Weak entity	New table with owner's PK as FK + partial key; PK = (owner PK, partial key). ON DELETE CASCADE.
1:1 relationship	Put the PK of one side as a UNIQUE foreign key in the other (prefer the side with total participation); or merge tables.
1:N relationship	Put the PK of the "1" side as a foreign key in the "N" side table.
M:N relationship	Create a junction table with the PKs of both entities as FKs (composite PK) + relationship attributes.
Ternary relationship	Junction table with FKs of all three entities.
Specialization	Options: (a) one table per class with PK shared, (b) one table per subclass only, (c) a single table with a type column.

4.8 Practical ERD Design Exercise - Library System

EXAMPLE

Requirements: A library has members. Members borrow books. Each book has one publisher and can have many authors; an author can write many books. A loan records the borrow date and return date.

Step	Result
1. Identify entities	MEMBER, BOOK, AUTHOR, PUBLISHER
2. Identify relationships	MEMBER borrows BOOK (M:N, attributes: BorrowDate, ReturnDate); AUTHOR writes BOOK (M:N); PUBLISHER publishes BOOK (1:N)
3. Add attributes and keys	MEMBER(MemberID, Name, Phone) • BOOK(ISBN, Title, Year) • AUTHOR(AuthorID, Name) • PUBLISHER(PubID, Name, City)
4. Convert to tables	BOOK(ISBN, Title, Year, PubID) • LOAN(MemberID, ISBN, BorrowDate, ReturnDate) • BOOK_AUTHOR(ISBN, AuthorID)

SQL - JUNCTION TABLE FOR THE M:N RELATIONSHIP

```sql
CREATE TABLE loan (
  member_id   INT,
  isbn        VARCHAR(13),
  borrow_date DATE,
  return_date DATE,
  PRIMARY KEY (member_id, isbn, borrow_date),
  FOREIGN KEY (member_id) REFERENCES member(member_id),
  FOREIGN KEY (isbn)      REFERENCES book(isbn)
);
```

EXAM TIP

ERD checklist: (1) every entity has a key; (2) every M:N is resolved with a junction table; (3) no multivalued attribute is left in a table; (4) relationships are named with verbs and show cardinality and participation.

TOPIC 5

Relational Database Model

5.1 Introduction to Relational Databases

DEFINITION

The **relational model** (E. F. Codd, 1970) organises data into **relations** (tables). Relationships are expressed by matching values in key columns, not by physical pointers. It is the foundation of SQL and almost all modern business systems.

5.2 Basic Terminology

Formal term	Everyday term	Meaning
Relation	Table	A set of tuples with the same attributes.
Tuple	Row / record	One entry in the table (one entity instance).
Attribute	Column / field	A named property with a domain.
Domain	Data type + allowed values	Set of valid values for an attribute (e.g. Age: integers 0-120).
Degree	-	Number of attributes (columns) of the relation.
Cardinality	-	Number of tuples (rows) in the relation.
Relational schema	Table structure	R(A1, A2, ..., An) - relation name + attributes with domains.
Relational instance	Table contents	Set of tuples in the relation at a given time.

EXAMPLE

STUDENT (StudentID, Name, Age, Course) has **degree = 4**. If it holds 250 rows its **cardinality = 250**.

WATCH OUT

A relation is a **set**: (1) no duplicate tuples, (2) row order does not matter, (3) column order does not matter, (4) every value is atomic (single), (5) each column has a unique name.

5.3 Types of Keys

EXAMPLE

Example relation: **EMPLOYEE(EmpID, NRC, Email, Name, DeptID)**

Key	Definition	In EMPLOYEE
Super key	Any set of attributes that uniquely identifies a tuple (may contain extra attributes).	{EmpID}, {EmpID, Name}, {NRC, Name}, {Email}...
Candidate key	A **minimal** super key - no attribute can be removed and uniqueness remains.	{EmpID}, {NRC}, {Email}
Primary key	The candidate key **chosen** to identify rows. Unique and NOT NULL; only one per table.	EmpID
Alternate key	Candidate keys **not** chosen as the primary key.	NRC, Email (declared UNIQUE)
Composite key	A key made of **two or more** attributes.	(StudentID, CourseCode) in ENROLMENT
Foreign key	An attribute that references the primary key of another (or the same) table; links tables.	DeptID → DEPARTMENT.DeptID
Surrogate key	An artificial, system-generated key with no business meaning (auto-increment / UUID).	EmpID as AUTO_INCREMENT

Key	Definition	In EMPLOYEE
Natural key	A key with real-world meaning.	NRC number, ISBN

KEY POINT
Hierarchy: **Super keys ⊃ Candidate keys ⊃ Primary key**. Every candidate key is a super key; not every super key is a candidate key.

5.4 Integrity Constraints

Constraint	Rule	Violated when...
Domain constraint	Values must come from the attribute's domain / type; CHECK rules.	Age = -5 or "abc"
Key (uniqueness) constraint	No two tuples share the same key value.	Two students with StudentID 101
Entity integrity	The **primary key cannot be NULL** (and must be unique).	Inserting a row with StudentID = NULL
Referential integrity	A foreign key must either be NULL or match an existing primary key value in the referenced table.	Enrolment for StudentID 999 that does not exist
NOT NULL / Business rules	Mandatory columns, or rules enforced by CHECK and triggers.	Balance < 0 on a savings account

Referential actions (what happens to child rows)

Action	On DELETE / UPDATE of the parent row
RESTRICT / NO ACTION	Reject the operation if child rows exist (default).
CASCADE	Automatically delete / update the matching child rows.
SET NULL	Set the child foreign key to NULL.
SET DEFAULT	Set the child foreign key to its default value.

SQL - ALL CONSTRAINTS IN ONE TABLE

```sql
CREATE TABLE student (
  student_id INT PRIMARY KEY,
  name       VARCHAR(50) NOT NULL,
  email      VARCHAR(80) UNIQUE,
  age        INT CHECK (age BETWEEN 15 AND 80),
  dept_id    INT,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
      ON DELETE SET NULL ON UPDATE CASCADE
);
```

PART 3 OF 10

Database Normalization

Designing good tables: functional dependencies, the normal forms 1NF to 6NF, decomposition and when to denormalize.

3

IN THIS PART

Topic 6 Functional Dependencies
Topic 7 Database Normalization

TOPIC 6

Functional Dependencies

6.1 Introduction

DEFINITION

A **functional dependency (FD) X → Y** means: if two tuples have the same value for attribute set X, they must have the same value for Y. We say "X determines Y" or "Y is functionally dependent on X". X is called the **determinant**.

EXAMPLE

StudentID → Name (a student ID determines exactly one name) | ISBN → Title | (BusNo, TravelDate) → Driver.
But Name → StudentID is **false** because two students may share a name.

6.2 Types of Functional Dependencies

Type	Definition	Example
Trivial FD	X → Y where Y ⊆ X (the right side is already inside the left side). Always true.	{StudentID, Name} → Name
Non-trivial FD	X → Y where Y ⊄ X (at least one attribute of Y is not in X).	StudentID → Name
Completely non-trivial	X → Y and X ∩ Y = ∅.	StudentID → Name
Full functional dependency	Y depends on **all** of a composite X and on no proper subset of it.	(StudentID, CourseID) → Grade
Partial dependency	Y depends on **only part** of a composite key. Violates 2NF.	(StudentID, CourseID) → StudentName, because StudentID → StudentName
Transitive dependency	X → Y and Y → Z (Y not a key) so X → Z indirectly. Violates 3NF.	EmpID → DeptID → DeptName
Multivalued dependency	X ↠ Y: each X value has a set of Y values independent of other attributes. Used in 4NF.	Employee ↠ Skill

6.3 Armstrong's Axioms and Derived Rules

Rule	Statement	Type
Reflexivity	If $Y \subseteq X$ then $X \rightarrow Y$	Primary axiom
Augmentation	If $X \rightarrow Y$ then $XZ \rightarrow YZ$ for any Z	Primary axiom
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$	Primary axiom
Union	If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$	Derived
Decomposition (Projectivity)	If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$	Derived
Pseudo-transitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$ then $WX \rightarrow Z$	Derived

KEY POINT
Armstrong's axioms are **sound** (they derive only valid FDs) and **complete** (they derive every FD implied by F). The set of all FDs implied by F is the **closure F^+**.

6.4 Attribute Closure (X^+)

DEFINITION
The **closure of X**, written X^+, is the set of all attributes that X functionally determines under a set of FDs F.

Algorithm	
1	Start with result = X.
2	Find any FD $A \rightarrow B$ in F where $A \subseteq$ result; then add B to result.
3	Repeat step 2 until no new attributes can be added.
4	The final result is X^+. If X^+ contains **all** attributes of R, X is a super key.

EXAMPLE
R(A, B, C, D, E) with F = { $A \rightarrow B$, $BC \rightarrow D$, $D \rightarrow E$ }. Find $(AC)^+$:
Start {A, C} → A → B gives {A, B, C} → BC → D gives {A, B, C, D} → D → E gives {A, B, C, D, E}.
$(AC)^+$ = ABCDE = all attributes, so AC is a super key (and a candidate key because neither A^+ nor C^+ is complete).

6.5 Minimal Cover (Canonical Cover)

DEFINITION
A **minimal cover** F_c is a smallest equivalent set of FDs: (1) every right side is a single attribute, (2) no extraneous attribute on any left side, (3) no redundant FD.

Step	Action
1. Split RHS	Rewrite so each FD has one attribute on the right (use decomposition).
2. Remove extraneous LHS attributes	For each FD with a composite LHS, test if an attribute can be removed without changing the closure.
3. Remove redundant FDs	Delete an FD if it can be derived from the others.

EXAMPLE
F = { $A \rightarrow BC$, $B \rightarrow C$, $A \rightarrow B$, $AB \rightarrow C$ }.
Step 1: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $A \rightarrow B$, $AB \rightarrow C$ → remove duplicates.
Step 2: in $AB \rightarrow C$, B is extraneous because $A \rightarrow C$ already holds, so it becomes $A \rightarrow C$.
Step 3: $A \rightarrow C$ is redundant ($A \rightarrow B$ and $B \rightarrow C$ give it).
Minimal cover = { $A \rightarrow B$, $B \rightarrow C$ }.

6.6 Candidate Key Identification

Step	Procedure
1	List attributes that **never appear on any right side** of an FD - they must be in every candidate key.
2	Compute the closure of that set. If it covers all attributes → it is the (only) candidate key.
3	Otherwise add attributes that appear on the right sides one by one (or in combinations) and test closures.
4	Keep only **minimal** sets whose closure is the full relation. These are the candidate keys.

EXAM TIP
Attributes that appear **only on the left** → always in the key. Attributes that appear **only on the right** → never in the key. Attributes on **neither side** → must be in the key.

TOPIC 7

Database Normalization

7.1 Definition, Importance and Objectives

DEFINITION
Normalization is the step-by-step process of organising tables to **reduce data redundancy** and **eliminate anomalies** by decomposing large tables into smaller, well-structured ones linked by keys. It was introduced by E. F. Codd.

Objectives / Importance	
Eliminate redundancy	Store each fact only once, saving storage.
Avoid update, insertion and deletion anomalies	Keeps data consistent and reliable.
Improve integrity	Dependencies are enforced through keys.
Make design logical	Each table describes one subject.
Make the database easier to extend and maintain	Changes affect fewer places.

7.2 Data Redundancy and Database Anomalies

DEFINITION
Data redundancy means the same data is stored unnecessarily in many places. It causes wasted space and three kinds of **anomalies**.

EmpID	EmpName	DeptID	DeptName	DeptHead
E1	Mary	D1	Computing	Dr. Banda
E2	John	D1	Computing	Dr. Banda
E3	Ruth	D2	Finance	Mr. Phiri
E4	Peter	D1	Computing	Dr. Banda

Anomaly	Meaning	Example in the table above
Insertion anomaly	Cannot add new data without adding unrelated data.	Cannot add a new department "Law" until at least one employee belongs to it.
Update anomaly	Changing one fact requires changing many rows; missing one gives inconsistency.	If Dr. Banda is replaced, three rows must be updated.
Deletion anomaly	Deleting a row unintentionally removes other facts.	Deleting Ruth (E3) also deletes the fact that Finance exists and Mr. Phiri heads it.

7.3 The Normal Forms at a Glance



Normal form	Requirement (in addition to the previous form)	Removes
1NF	All attributes atomic; no repeating groups; each row unique (has a key).	Multivalued / repeating groups
2NF	In 1NF + **no partial dependency** (non-key attributes depend on the *whole* key).	Partial dependencies
3NF	In 2NF + **no transitive dependency** (non-key attributes depend only on the key).	Transitive dependencies
BCNF	For every non-trivial FD X → Y, **X is a super key**.	Anomalies from overlapping candidate keys
4NF	In BCNF + no non-trivial **multivalued dependencies**.	Independent multivalued facts
5NF (PJNF)	In 4NF + no non-trivial **join dependencies** (cannot be split further losslessly).	Join dependencies
6NF	Every join dependency is trivial; table = key + at most one other attribute.	Temporal / historical redundancy

EXAM TIP
Memory aid for 1NF-3NF: every non-key attribute must depend on **"the key (1NF), the whole key (2NF), and nothing but the key (3NF)"** - so help me Codd.

7.4 First Normal Form (1NF)

- Each cell holds **one atomic value** - no lists or sets in a column.
- No repeating groups or arrays (Phone1, Phone2, Phone3).
- Each record is unique and the table has a primary key.

StudentID	Name	Phones
1	Mary	0971, 0955
2	John	0966

StudentID	Name	Phone
1	Mary	0971
1	Mary	0955
2	John	0966

GOOD TO KNOW
Better still: put phones in a separate table **STUDENT_PHONE(StudentID, Phone)** to avoid repeating Name.

7.5 Second Normal Form (2NF)

DEFINITION
A relation is in **2NF** if it is in 1NF and **no non-prime attribute is partially dependent** on any candidate key. (Only matters when the key is composite.)

EXAMPLE
ENROL(StudentID, CourseID, StudentName, CourseTitle, Grade) - key (StudentID, CourseID)
StudentID → StudentName and CourseID → CourseTitle are **partial dependencies**.
2NF decomposition: STUDENT(StudentID, StudentName) • COURSE(CourseID, CourseTitle) • ENROL(StudentID, CourseID, Grade)

7.6 Third Normal Form (3NF)

DEFINITION
A relation is in **3NF** if it is in 2NF and has **no transitive dependency** of a non-prime attribute on the key. Formally, for every FD X → A: either X is a super key, **or** A is a prime attribute (part of some candidate key).

EXAMPLE
EMPLOYEE(EmpID, Name, DeptID, DeptName) - EmpID → DeptID → DeptName (transitive).
3NF decomposition: EMPLOYEE(EmpID, Name, DeptID) • DEPARTMENT(DeptID, DeptName)

7.7 Boyce-Codd Normal Form (BCNF)

DEFINITION
A relation is in **BCNF** if, for every non-trivial FD X → Y, **X is a super key**. BCNF is stricter than 3NF - it removes the "or A is prime" loophole.

EXAMPLE
TEACH(Student, Subject, Teacher) with FDs: (Student, Subject) → Teacher; **Teacher → Subject** (each teacher teaches one subject).
Candidate keys: (Student, Subject) and (Student, Teacher). It is in 3NF (Subject is prime) **but not BCNF** because Teacher is not a super key.
BCNF decomposition: TEACHER_SUBJECT(Teacher, Subject) • STUDENT_TEACHER(Student, Teacher)

WATCH OUT
BCNF decomposition is always lossless but may **not preserve all dependencies**. 3NF decomposition can always be both lossless and dependency-preserving.

7.8 Fourth, Fifth and Sixth Normal Forms

Form	Problem it solves	Example / solution
4NF	Multivalued dependency X ↠ Y | Z: two **independent** multivalued facts stored in one table force every combination to be stored.	EMP_SKILL_LANG(Employee, Skill, Language) where Employee ↠ Skill | Language. Split into EMP_SKILL(Employee, Skill) and EMP_LANG(Employee, Language).
5NF (PJNF)	A table that can only be reconstructed by joining **three or more** projections (join dependency) although no 2-way split works.	SUPPLIER_PART_PROJECT(S, P, J): decompose into SP(S, P), PJ(P, J), SJ(S, J); the three-way join gives the original back losslessly.
6NF	Temporal data: a record changes over time and attributes change independently.	Each table = key + time period + **one** attribute (e.g. EMP_SALARY(EmpID, ValidFrom, Salary)). Used in anchor modelling and data warehouses.

7.9 Lossless Decomposition and Dependency Preservation

Property	Meaning	Test
Lossless-join decomposition	Joining the decomposed tables gives back **exactly** the original table - no lost or spurious tuples.	For R decomposed into R1 and R2: (R1 ∩ R2) → R1 **or** (R1 ∩ R2) → R2 must hold (common attributes form a key of one part).
Dependency preservation	Every original FD can be checked within a single decomposed table, without joins.	$(F1 \cup F2 \cup \ldots)^+ = F^+$

KEY POINT
A good decomposition must be **lossless** (non-negotiable) and ideally **dependency preserving**. 3NF can guarantee both; BCNF guarantees only lossless.

7.10 Worked Example: Normalizing Step by Step

EXAMPLE
Unnormalized: ENROL(StudentID, StudentName, CourseID, CourseTitle, Instructor, InstructorOffice, Grade)
FDs: (StudentID, CourseID) → Grade; StudentID → StudentName; CourseID → CourseTitle, Instructor; Instructor → InstructorOffice

Stage	Problem found	Result
1NF	Already atomic with key (StudentID, CourseID)	No change
2NF	StudentName depends only on StudentID; CourseTitle, Instructor, Office depend only on CourseID (partial dependencies)	STUDENT(StudentID, StudentName) • COURSE(CourseID, CourseTitle, Instructor, InstructorOffice) • ENROL(StudentID, CourseID, Grade)
3NF	In COURSE: CourseID → Instructor → InstructorOffice (transitive)	COURSE(CourseID, CourseTitle, InstructorID) • INSTRUCTOR(InstructorID, Name, Office)
BCNF	All determinants are keys	Final 4 tables - no further change

7.11 Denormalization

DEFINITION
Denormalization is the deliberate introduction of controlled redundancy (merging tables, storing derived values or duplicating columns) to **improve read performance**, usually in reporting systems and data warehouses.

Use denormalization when...	Risks
Queries need many expensive joins	More storage used
Read-heavy workloads / reporting / OLAP	Update, insert and delete anomalies return
Pre-computed totals and summaries are needed	Extra code (triggers) needed to keep copies in sync
Data is rarely changed (e.g. data warehouse)	Less flexible design

7.12 Advantages and Disadvantages of Normalization

Advantages	Disadvantages
Minimal redundancy and storage	More tables and more joins → slower complex queries
No update, insert or delete anomalies	Harder for beginners to read and design
Better data integrity and consistency	Queries become longer and more complex
Easier maintenance and smaller, focused tables	Over-normalization hurts read performance
Flexible: easy to add new data types or relationships	Needs thorough analysis of dependencies

7.13 Practical Normalization Exercises

EXAMPLE
Exercise 1 - Invoice: INVOICE(InvNo, InvDate, CustID, CustName, CustCity, ProdID, ProdName, UnitPrice, Qty).
Key: (InvNo, ProdID). FDs: InvNo → InvDate, CustID; CustID → CustName, CustCity; ProdID → ProdName, UnitPrice; (InvNo, ProdID) → Qty.
Answer (3NF): INVOICE(InvNo, InvDate, CustID) • CUSTOMER(CustID, CustName, CustCity) • PRODUCT(ProdID, ProdName, UnitPrice) • INVOICE_LINE(InvNo, ProdID, Qty)

EXAMPLE

Exercise 2 - Bus booking: BOOKING(BookingID, PassengerName, PassengerPhone, BusNo, Route, DepartTime, SeatNo, Fare).
Hint: PassengerName/Phone depend on the passenger; Route and DepartTime on the trip; Fare on the route. Separate PASSENGER, TRIP, ROUTE, BUS and BOOKING (BookingID, PassengerID, TripID, SeatNo).

EXAM TIP

Exam method: (1) list FDs, (2) find candidate keys using closure, (3) check each normal form in order, (4) decompose violators, (5) verify lossless join and dependency preservation.

PART 4 OF 10

Structured Query Language (SQL)

4

The standard language of relational databases: syntax, command categories, everyday queries, joins and advanced features.

IN THIS PART

Topic 8	Introduction to SQL
Topic 9	SQL Command Categories
Topic 10	SQL Practical Operations
Topic 11	SQL Joins
Topic 12	Advanced SQL

TOPIC 8

Introduction to SQL

8.1 Meaning and Importance of SQL

DEFINITION

SQL (Structured Query Language) is the standard, **declarative** language for defining, manipulating, querying and controlling data in relational databases. You say *what* you want, and the DBMS decides *how* to get it. Developed at IBM in the 1970s (SEQUEL); standardised by ANSI (1986) and ISO (1987).

- Universal - works across MySQL, PostgreSQL, Oracle, SQL Server, SQLite
- Easy to learn, English-like syntax
- Handles huge volumes of data efficiently
- Used by developers, analysts, data scientists and DBAs
- Supports security, transactions and integrity
- Foundation for BI, reporting and data science

8.2 SQL Syntax Basics

Rule	Detail
Statements	End with a semicolon `;` (required in most tools to separate statements).
Keywords	Not case-sensitive (`select` = SELECT); convention: UPPERCASE keywords, lowercase names.
Identifiers	Table / column names: letters, digits, underscore; avoid spaces and reserved words.
Strings and dates	Single quotes: `'Mary'`, `'2026-10-15'` (ISO format YYYY-MM-DD).
Order of clauses	SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT.

KEY POINT

Logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This is why a column alias defined in SELECT cannot be used in WHERE.

8.3 SQL Data Types

Category	Types	Notes
Integer	TINYINT, SMALLINT, INT, BIGINT	Whole numbers; INT is 4 bytes
Exact numeric	DECIMAL(p,s), NUMERIC(p,s)	Money: DECIMAL(10,2) - precise
Approximate	FLOAT, REAL, DOUBLE	Scientific; rounding errors possible
Character	CHAR(n), VARCHAR(n), TEXT	CHAR fixed length; VARCHAR variable
Date / Time	DATE, TIME, DATETIME, TIMESTAMP, YEAR	Dates, schedules, logs
Boolean	BOOLEAN / TINYINT(1)	TRUE / FALSE
Binary / Large	BLOB, BINARY	Images, files
Other	ENUM, JSON, UUID	Allowed-value lists; semi-structured data

8.4 SQL Operators

Type	Operators	Example
Arithmetic	+ - * / %	`price * qty`
Comparison	= <> (or !=) < > <= >=	`age >= 18`
Logical	AND OR NOT	`age > 18 AND city = 'Ndola'`
Special	BETWEEN, IN, LIKE, IS NULL, EXISTS, ANY, ALL	`name LIKE 'M%'`
String	CONCAT() or ||	`CONCAT(first, ' ', last)`
Set	UNION, INTERSECT, EXCEPT	Combine result sets

8.5 Statements, Comments and Constraints

SQL - COMMENTS AND A STATEMENT

```sql
-- single-line comment
/* multi-line
   comment */
SELECT name, age FROM student;  -- a statement
```

Constraint	Purpose	Example
PRIMARY KEY	Unique + NOT NULL row identifier	id INT PRIMARY KEY
FOREIGN KEY	Links to a parent table (referential integrity)	FOREIGN KEY (dept_id) REFERENCES dept(id)
UNIQUE	No duplicates (NULL allowed)	email VARCHAR(80) UNIQUE
NOT NULL	Value is required	name VARCHAR(50) NOT NULL
CHECK	Value must satisfy a condition	CHECK (age >= 18)
DEFAULT	Value used when none supplied	status VARCHAR(10) DEFAULT 'active'
AUTO_INCREMENT	Auto-generated numbers (MySQL)	id INT AUTO_INCREMENT

TOPIC 9

SQL Command Categories

Category	Full meaning	Purpose	Common commands
DDL	Data Definition Language	Defines / changes structure (schema). Auto-commits in most systems.	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Changes the data stored in tables.	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieves data without changing it.	SELECT
DCL	Data Control Language	Controls permissions and access.	GRANT, REVOKE
TCL	Transaction Control Language	Manages groups of statements as transactions.	COMMIT, ROLLBACK, SAVEPOINT

Command	Category	What it does	Can ROLLBACK?
DELETE	DML	Removes selected rows (WHERE), logged row by row	Yes
TRUNCATE	DDL	Removes **all** rows quickly; resets identity; keeps structure	Usually no
DROP	DDL	Removes the whole table / object (structure + data)	No
ALTER	DDL	Adds, modifies or drops columns / constraints	No

EXAM TIP

DELETE vs TRUNCATE vs DROP: DELETE = remove some or all rows (DML); TRUNCATE = empty the table fast (DDL); DROP = destroy the table itself (DDL).

SQL - ONE EXAMPLE OF EACH CATEGORY

```sql
-- DDL
CREATE TABLE student (id INT PRIMARY KEY, name VARCHAR(50), age INT);
ALTER TABLE student ADD email VARCHAR(80);
-- DML
INSERT INTO student (id, name, age) VALUES (1, 'Mary', 20);
-- DQL
SELECT * FROM student;
-- DCL
GRANT SELECT ON student TO clerk;
-- TCL
COMMIT;
```

TOPIC 10

SQL Practical Operations

GOOD TO KNOW

Sample tables used below: **student(id, name, age, dept, gpa)** and **course(code, title, credits)**.

10.1 Creating Databases and Tables

```sql
CREATE DATABASE school;
USE school;

CREATE TABLE student (
  id    INT PRIMARY KEY AUTO_INCREMENT,
  name  VARCHAR(50) NOT NULL,
  age   INT,
  dept  VARCHAR(30),
  gpa   DECIMAL(3,2) DEFAULT 0.00
);
ALTER TABLE student ADD COLUMN email VARCHAR(80);
ALTER TABLE student DROP COLUMN email;
DROP TABLE student;      -- DANGEROUS: removes table and data
DROP DATABASE school;
```

10.2 INSERT, SELECT, UPDATE, DELETE

```sql
INSERT INTO student (name, age, dept, gpa) VALUES ('Mary', 20, 'Computing', 3.6);
INSERT INTO student (name, age, dept, gpa)
VALUES ('John', 22, 'Finance', 3.1), ('Ruth', 21, 'Computing', NULL);

SELECT * FROM student;                    -- all columns
SELECT name, gpa FROM student;            -- chosen columns

UPDATE student SET gpa = 3.8 WHERE name = 'Mary';
DELETE FROM student WHERE id = 3;          -- always use WHERE!
```

WATCH OUT

Running **UPDATE** or **DELETE** without a **WHERE** clause changes or removes **every** row. Test with SELECT first and use transactions.

10.3 Filtering, Sorting and Limiting

Feature	Syntax	Meaning
WHERE	`WHERE dept = 'Computing' AND age > 20`	Filter rows before grouping
ORDER BY	`ORDER BY gpa DESC, name ASC`	Sort (ASC default)
DISTINCT	`SELECT DISTINCT dept FROM student`	Remove duplicate rows
LIMIT / OFFSET	`LIMIT 5 OFFSET 10`	Return 5 rows after skipping 10 (paging). SQL Server: TOP / FETCH
BETWEEN	`age BETWEEN 18 AND 25`	Inclusive range
IN	`dept IN ('Computing','Law')`	Matches any value in a list
LIKE	`name LIKE 'M%'`	**%** = any characters, _ = exactly one character
IS NULL	`gpa IS NULL`	NULL tests - never use "= NULL"
Alias	`SELECT name AS full_name`	Temporary readable name

WATCH OUT

NULL means unknown / missing - not zero or empty. **NULL = NULL** is not true. Any arithmetic with NULL gives NULL. Use **IS NULL**, **COALESCE(x, 0)** or **IFNULL**.

```sql
SELECT name, gpa FROM student
WHERE dept = 'Computing' AND gpa IS NOT NULL
ORDER BY gpa DESC
LIMIT 3;
```

10.4 Aggregate Functions, GROUP BY and HAVING

Function	Returns	Notes
COUNT(*)	Number of rows	COUNT(col) skips NULLs; COUNT(DISTINCT col) counts unique
SUM(col)	Total	Ignores NULLs
AVG(col)	Average	Ignores NULLs
MIN(col) / MAX(col)	Smallest / largest	Works on numbers, text, dates

```sql
SELECT dept, COUNT(*) AS students, AVG(gpa) AS avg_gpa
FROM student
WHERE age >= 18             -- filters ROWS before grouping
GROUP BY dept
HAVING AVG(gpa) > 3.0       -- filters GROUPS after aggregation
ORDER BY avg_gpa DESC;
```

KEY POINT

WHERE filters rows *before* grouping and cannot use aggregates. **HAVING** filters groups *after* aggregation. Every non-aggregated column in SELECT must appear in GROUP BY.

10.5 Subqueries, Nested Queries and CTEs

Type	Description	Example
Scalar subquery	Returns one value	`WHERE gpa > (SELECT AVG(gpa) FROM student)`
Multi-row subquery	Returns a list; used with IN / ANY / ALL	`WHERE id IN (SELECT id FROM enrol)`
Correlated subquery	Refers to the outer query; runs per outer row	`WHERE EXISTS (SELECT 1 FROM enrol e WHERE e.id = s.id)`
Derived table	Subquery in FROM (needs an alias)	`FROM (SELECT ...) AS t`

```sql
-- students who scored above the class average
SELECT name FROM student
WHERE gpa > (SELECT AVG(gpa) FROM student);

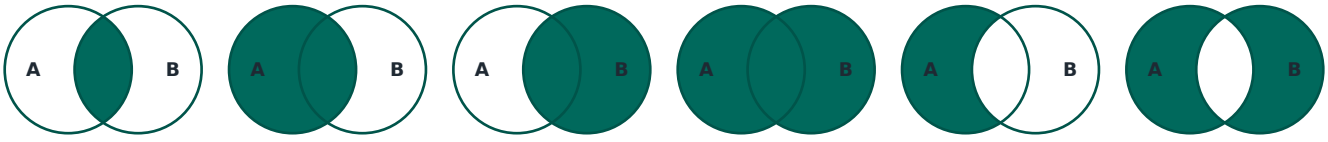
-- Common Table Expression (CTE): a named, temporary result for one query
WITH dept_avg AS (
  SELECT dept, AVG(gpa) AS avg_gpa FROM student GROUP BY dept
)
SELECT s.name, s.gpa, d.avg_gpa
FROM student s JOIN dept_avg d ON s.dept = d.dept
WHERE s.gpa > d.avg_gpa;
```

TOPIC 11

SQL Joins

DEFINITION

A **join** combines rows from two or more tables based on a related column (usually foreign key = primary key). Joins let a normalized database be queried as if it were one table.



Student (A)	DeptID
Mary	1
John	2
Ruth	NULL

Dept (B)	DeptID
Computing	1
Finance	2
Law	3

Join type	Returns	Result using the tables above
INNER JOIN	Only rows with a **match in both** tables	Mary-Computing, John-Finance
LEFT (OUTER) JOIN	All rows from the left + matches from right (NULL if none)	Mary, John, Ruth (Ruth has NULL dept)
RIGHT (OUTER) JOIN	All rows from the right + matches from left	Mary, John, and Law (no student)
FULL OUTER JOIN	All rows from both; NULL where no match (MySQL: LEFT UNION RIGHT)	Mary, John, Ruth, Law
CROSS JOIN	Cartesian product: every row with every row (m × n)	3 × 3 = 9 rows
SELF JOIN	Table joined to itself using aliases	Employee and their manager
NATURAL JOIN	Joins on all columns with identical names (risky)	-

SQL - JOIN EXAMPLES

```sql
SELECT s.name, d.dept_name
FROM student s
INNER JOIN dept d ON s.dept_id = d.dept_id;

SELECT s.name, d.dept_name           -- keep students with no department
FROM student s LEFT JOIN dept d ON s.dept_id = d.dept_id;

SELECT d.dept_name                   -- departments with NO students (anti-join)
FROM dept d LEFT JOIN student s ON s.dept_id = d.dept_id
WHERE s.id IS NULL;

SELECT e.name AS employee, m.name AS manager   -- self join
FROM employee e LEFT JOIN employee m ON e.manager_id = m.emp_id;

-- joining multiple tables
SELECT s.name, c.title, e.grade
FROM enrol e
JOIN student s ON e.student_id = s.id
JOIN course  c ON e.course_code = c.code;
```

Joins vs Subqueries

	Joins	Subqueries
Columns returned	Can show columns from all tables	Only from the outer query
Readability	Good for combining data	Good for step-by-step logic and existence checks
Performance	Usually faster (optimizer handles well)	Correlated subqueries can be slow; modern optimizers often rewrite them
Best for	Combining columns from related tables	Filtering by aggregates, EXISTS / NOT EXISTS, IN lists

EXAM TIP
Practice: (1) list every student and their course titles; (2) list courses with no enrolments; (3) count students per department including departments with zero students (LEFT JOIN + COUNT of the right-side column).

TOPIC 12

Advanced SQL

12.1 Views

DEFINITION
A **view** is a saved SELECT query that behaves like a virtual table. It stores no data (except materialized views). Uses: simplify complex queries, hide sensitive columns (security), provide logical data independence.

```sql
CREATE VIEW computing_students AS
SELECT id, name, gpa FROM student WHERE dept = 'Computing';

SELECT * FROM computing_students;
DROP VIEW computing_students;
```

12.2 Stored Procedures, Functions and Triggers

Object	Description	How it runs	Returns
Stored procedure	Named, precompiled block of SQL stored in the DB; accepts IN / OUT parameters	Explicitly: `CALL proc()`	Optional
User-defined function	Reusable routine usable inside queries	In SELECT / WHERE	Always a value (scalar or table)
Trigger	Code that fires **automatically** on INSERT / UPDATE / DELETE (BEFORE or AFTER)	Automatically by events	Nothing

SQL - MYSQL SYNTAX

```
DELIMITER //
CREATE PROCEDURE raise_gpa(IN p_id INT, IN p_amount DECIMAL(3,2))
BEGIN
  UPDATE student SET gpa = gpa + p_amount WHERE id = p_id;
END //

CREATE FUNCTION full_name(f VARCHAR(30), l VARCHAR(30)) RETURNS VARCHAR(61)
DETERMINISTIC
BEGIN
  RETURN CONCAT(f, ' ', l);
END //

CREATE TRIGGER log_delete AFTER DELETE ON student
FOR EACH ROW
BEGIN
  INSERT INTO audit_log(action, student_id, at) VALUES ('DELETE', OLD.id, NOW());
END //
DELIMITER ;
CALL raise_gpa(1, 0.10);
```

GOOD TO KNOW

Benefits of stored routines: faster (precompiled), reusable, centralised business logic, better security (grant EXECUTE only), less network traffic. Triggers are great for **auditing** and enforcing complex rules, but hidden logic can be hard to debug.

12.3 Cursors and Temporary Tables

Feature	Description	Notes
Cursor	Processes query results **row by row** inside procedures: DECLARE → OPEN → FETCH (loop) → CLOSE	Slow - prefer set-based SQL when possible
Temporary table	Table that exists only for the session / procedure: `CREATE TEMPORARY TABLE tmp AS SELECT ...`	Auto-dropped at session end; useful for intermediate results

12.4 Recursive Queries

```
SQL
-- all employees under manager 1 (organisation chart)
WITH RECURSIVE chain AS (
  SELECT emp_id, name, manager_id, 1 AS level
  FROM employee WHERE emp_id = 1                -- anchor
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, c.level + 1
  FROM employee e JOIN chain c ON e.manager_id = c.emp_id   -- recursive part
)
SELECT * FROM chain;
```

12.5 Window Functions

DEFINITION

A **window function** computes a value across a set of related rows (the window) **without collapsing them** into one row like GROUP BY does. Syntax: `function() OVER (PARTITION BY ... ORDER BY ...)`

Function	Behaviour	Scores 90, 90, 80
ROW_NUMBER()	Unique sequential number; ties get different numbers	1, 2, 3
RANK()	Same rank for ties; **gaps** follow	1, 1, 3
DENSE_RANK()	Same rank for ties; **no gaps**	1, 1, 2
LEAD(col, n)	Value from the **next** n-th row	Compare with next month
LAG(col, n)	Value from the **previous** n-th row	Compare with last month
SUM() OVER	Running total / moving aggregate	Cumulative sales

Function	Behaviour	Scores 90, 90, 80
NTILE(n)	Split rows into n buckets	Quartiles

```sql
SELECT name, dept, gpa,
       RANK() OVER (PARTITION BY dept ORDER BY gpa DESC) AS dept_rank,
       gpa - LAG(gpa) OVER (ORDER BY id) AS change_from_prev
FROM student;
```

12.6 CASE, Set Operations and Dynamic SQL

```sql
SELECT name, gpa,
  CASE WHEN gpa >= 3.5 THEN 'Distinction'
       WHEN gpa >= 2.5 THEN 'Pass'
       ELSE 'Fail' END AS grade
FROM student;
```

Set operator	Result	Rules
UNION	Combines results, **removes duplicates**	Same number of columns, compatible types, column names from first query
UNION ALL	Combines results, **keeps duplicates** (faster)	Same as above
INTERSECT	Rows that appear in **both** results	Not in older MySQL (use JOIN / IN)
EXCEPT / MINUS	Rows in the first result but **not** in the second	Oracle uses MINUS

DEFINITION

Dynamic SQL builds and executes SQL text at run time (e.g. `PREPARE stmt FROM @sql; EXECUTE stmt;`). It is flexible, but **must use parameters** - concatenating user input causes SQL injection.

12.7 SQL Performance Optimization

Technique	Why it helps
Create indexes on columns used in WHERE, JOIN, ORDER BY	Avoids full table scans
Select only needed columns - avoid SELECT *	Less data read and transferred
Filter early with WHERE; avoid functions on indexed columns	`WHERE YEAR(d)=2026` disables an index - use a date range instead
Use EXPLAIN / EXPLAIN ANALYZE to inspect the execution plan	Find scans, bad joins, missing indexes
Prefer JOIN or EXISTS over deeply nested / correlated subqueries	Optimizer produces better plans
Use UNION ALL instead of UNION when duplicates do not matter	Skips costly duplicate removal
Paginate with LIMIT; batch large inserts	Smaller transactions, less memory
Normalize, but denormalize / cache hot read paths	Balance integrity and speed
Avoid leading wildcards `LIKE '%abc'`	Cannot use a normal B-tree index

PART 5 OF 10

Relational Algebra and Calculus

5

The mathematical foundations of SQL: procedural relational algebra and declarative relational calculus.

IN THIS PART

Topic 13 Relational Algebra
Topic 14 Relational Calculus

TOPIC 13

Relational Algebra

13.1 Introduction

DEFINITION

Relational algebra is a **procedural** query language: a set of operators that take one or two relations as input and produce a new relation as output. It is the theoretical basis for SQL and for query optimisation. Operators can be **composed** (closure property).

13.2 Fundamental and Derived Operations

Operation	Symbol	Notation	Meaning	SQL equivalent
Selection	σ	$\sigma_{condition}(R)$	Chooses **rows** that satisfy a condition	WHERE
Projection	π	$\pi_{A1,A2}(R)$	Chooses **columns**; removes duplicates	SELECT DISTINCT cols
Union	∪	R ∪ S	All tuples in R or S (duplicates removed)	UNION
Set difference	−	R − S	Tuples in R but not in S	EXCEPT / MINUS
Cartesian product	×	R × S	Every tuple of R paired with every tuple of S	CROSS JOIN
Rename	ρ	$\rho_x(R)$	Renames a relation or its attributes	AS
Intersection	∩	R ∩ S	Tuples in both R and S (= R − (R − S))	INTERSECT
Natural join	⋈	R ⋈ S	Join on all common attribute names	NATURAL JOIN
Theta join	$\bowtie_\theta$	R $\bowtie_\theta$ S	Product followed by selection with condition θ	JOIN ... ON
Division	÷	R ÷ S	Tuples of R related to **all** tuples of S	NOT EXISTS (double)

KEY POINT

Fundamental (minimal) set: σ, π, ∪, −, ×, ρ. All others (∩, ⋈, ÷) can be expressed using them. Union, intersection and difference require **union-compatible** relations (same number of attributes with compatible domains).

13.3 Worked Examples

sid	name	age	dept
1	Mary	20	CS
2	John	22	FIN
3	Ruth	21	CS

EXAMPLE
Relation **STUDENT** shown above.

Query in words	Relational algebra	Result
Students in CS	$\sigma_{dept='CS'}(STUDENT)$	Mary, Ruth rows
Names and ages only	$\pi_{name,age}(STUDENT)$	(Mary,20) (John,22) (Ruth,21)
Names of CS students older than 20	$\pi_{name}(\sigma_{dept='CS' \wedge age>20}(STUDENT))$	Ruth
Students with their course titles	$\pi_{name,title}(STUDENT \bowtie ENROL \bowtie COURSE)$	Name-title pairs
Students who took all courses	$\pi_{sid,cid}(ENROL) \div \pi_{cid}(COURSE)$	Division example
Students not enrolled in anything	$\pi_{sid}(STUDENT) - \pi_{sid}(ENROL)$	Set difference

13.4 Converting Relational Algebra into SQL

SQL EQUIVALENTS

```sql
-- pi_name( sigma_{dept='CS' AND age>20}(STUDENT) )
SELECT DISTINCT name FROM student WHERE dept = 'CS' AND age > 20;

-- pi_name,title( STUDENT natural-join ENROL natural-join COURSE )
SELECT DISTINCT s.name, c.title
FROM student s JOIN enrol e ON s.sid = e.sid JOIN course c ON e.cid = c.cid;

-- Division: students enrolled in ALL courses
SELECT s.sid FROM student s
WHERE NOT EXISTS (
  SELECT c.cid FROM course c
  WHERE NOT EXISTS (SELECT 1 FROM enrol e WHERE e.sid = s.sid AND e.cid = c.cid));
```

EXAM TIP
Remember the mapping: **σ → WHERE**, **π → SELECT**, **× → FROM A, B**, **⋈ → JOIN**, **ρ → AS**, **∪ → UNION**, **− → EXCEPT**. Relational algebra works on **sets** (no duplicates); SQL works on **bags** (duplicates allowed unless DISTINCT).

TOPIC 14

Relational Calculus

14.1 Introduction

DEFINITION
Relational calculus is a **declarative (non-procedural)** query language based on predicate logic. A query states *what* to retrieve, not *how*. Two forms exist: **Tuple Relational Calculus (TRC)** and **Domain Relational Calculus (DRC)**. SQL is mostly based on TRC; QBE is based on DRC.

14.2 Tuple vs Domain Relational Calculus

	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variables range over	Whole **tuples** (rows)	Individual **attribute values** (domains)
General form	{ t | P(t) }	{ <x1, x2, ..., xn> | P(x1, ..., xn) }
Example: CS students	{ t | STUDENT(t) ∧ t.dept = 'CS' }	{ <s, n, a, d> | STUDENT(s, n, a, d) ∧ d = 'CS' }
Names of students older than 20	{ t.name | STUDENT(t) ∧ t.age > 20 }	{ <n> | ∃s ∃a ∃d (STUDENT(s, n, a, d) ∧ a > 20) }
Closest to	SQL	QBE (Query By Example)

14.3 Free and Bound Variables, Quantifiers

Concept	Meaning
Free variable	A variable **not** bound by a quantifier; it appears to the left of "|" and represents the result of the query.
Bound variable	A variable that is under the scope of a quantifier (∃ or ∀) inside the formula.
Existential quantifier ∃	"There exists" at least one tuple / value such that the condition holds. ∃t (R(t) ∧ P(t)). Corresponds to IN / EXISTS.
Universal quantifier ∀	"For all" tuples the condition holds. ∀t (R(t) ⇒ P(t)). Corresponds to NOT EXISTS (NOT ...).
Logical connectives	∧ (and), ∨ (or), ¬ (not), ⇒ (implies).

KEY POINT
Equivalence: **∀x P(x) ≡ ¬∃x ¬P(x)**. This is why division uses a double NOT EXISTS in SQL.

14.4 Safe Expressions

WATCH OUT
An expression is **safe** if it produces a **finite** result drawn only from values present in the database. **Unsafe example:** { t | ¬ STUDENT(t) } - there are infinitely many tuples not in STUDENT. Safe calculus has the same expressive power as relational algebra.

14.5 Relational Algebra versus Relational Calculus

Feature	Relational Algebra	Relational Calculus
Style	Procedural - sequence of operations	Declarative - describes the result
Specifies	How to get the result	What the result is
Foundation	Set theory	First-order predicate logic
Expressive power	Equivalent (Codd's theorem) - safe calculus = algebra	Equivalent
Role	Internal representation for query optimisation	Basis for user-level languages (SQL, QBE)

EXAM TIP
Codd's Theorem: relational algebra and safe relational calculus have **equal expressive power**. A language that can express everything algebra can is called **relationally complete** (SQL is).

PART 6 OF 10

Transaction Management

6

Keeping data correct when many users read and write at the same time and when systems fail: transactions, ACID and concurrency control.

IN THIS PART

Topic 15 Database Transactions
Topic 16 Concurrency Control

TOPIC 15

Database Transactions

15.1 Meaning and Importance

DEFINITION

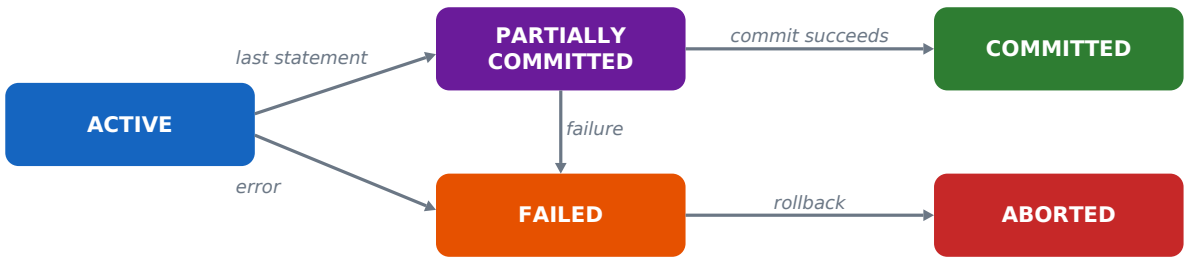
A **transaction** is a logical unit of work made of one or more database operations (reads / writes) that must be executed **entirely or not at all**. It moves the database from one consistent state to another.

EXAMPLE

Mobile money transfer of K100 from A to B: (1) read A's balance, (2) A = A − 100, (3) B = B + 100, (4) write both. If the system crashes after step 2, K100 would vanish - a transaction guarantees that either **both** updates happen or **neither**.

Transaction operation	Meaning
BEGIN / START TRANSACTION	Marks the start of a transaction
READ(X)	Copies data item X from the database into a program buffer
WRITE(X)	Writes the buffer value of X back to the database
COMMIT	Ends successfully; all changes become permanent
ROLLBACK (ABORT)	Ends unsuccessfully; all changes are undone
SAVEPOINT	Marks a point inside a transaction to which you can partially roll back

15.2 Transaction States



State	Meaning
Active	Initial state; the transaction is executing its operations.
Partially committed	The final statement has executed, but changes may still be only in memory buffers.
Committed	Changes are safely written to disk (log flushed); permanent. Cannot be rolled back.
Failed	Normal execution can no longer continue (error, crash, constraint violation).
Aborted	Rolled back; database restored to its state before the transaction. System may restart or kill it.

15.3 ACID Properties

Property	Meaning	Ensured by	Bank transfer example
Atomicity	"All or nothing" - the transaction is one indivisible unit.	Recovery manager (undo log, rollback)	Debit and credit both happen, or neither.
Consistency	The database moves from one valid state to another; all constraints and rules hold.	Integrity constraints, application logic	Total money in the system is unchanged; no negative balance.
Isolation	Concurrent transactions do not interfere; result equals some serial order.	Concurrency control (locks, MVCC)	Another user never sees money "in the middle" of the transfer.
Durability	Once committed, changes survive crashes and power loss.	Write-ahead log, redo, backups	After "Transfer successful", a crash cannot undo it.

SQL - TRANSACTION WITH SAVEPOINT

```sql
START TRANSACTION;
UPDATE account SET balance = balance - 100 WHERE acc_no = 'A';
SAVEPOINT after_debit;
UPDATE account SET balance = balance + 100 WHERE acc_no = 'B';
-- if something went wrong:   ROLLBACK TO after_debit;   (partial undo)
-- if everything is fine:
COMMIT;                -- permanent
-- or ROLLBACK;        -- undo everything
```

15.4 Schedules: Serial and Concurrent Execution

Term	Meaning
Schedule	The chronological order in which operations of several transactions are executed.
Serial schedule	Transactions run **one after another** with no interleaving. Always correct but slow (no concurrency).
Concurrent (non-serial) schedule	Operations of different transactions are **interleaved** for better throughput and resource use. May cause problems unless controlled.
Serializable schedule	A concurrent schedule whose effect equals that of **some serial schedule** - the correctness criterion.
Recoverable schedule	A transaction commits only after all transactions whose data it read have committed.
Cascadeless schedule	Transactions read only data written by already **committed** transactions, preventing cascading rollbacks.

TOPIC 16

Concurrency Control

16.1 Why Concurrency Control?

DEFINITION

Concurrency control manages simultaneous execution of transactions so that they appear to run in isolation, preserving consistency while keeping performance high.

16.2 Concurrency Problems

Problem	What happens	Example
Lost update	Two transactions read the same value and both write; one update overwrites the other.	Balance 500. T1 adds 100, T2 adds 50 at the same time; final balance 550 or 600 instead of 650.
Dirty read (uncommitted dependency)	T2 reads data written by T1 that is **not yet committed**; T1 then rolls back.	T2 reports a seat as booked, but T1 cancels.
Non-repeatable read	T1 reads a row twice and gets **different values** because T2 modified and committed it between reads.	Price is 10 then 12 inside one report.
Phantom read	T1 runs the same query twice and the second time sees **new rows** inserted by T2.	COUNT(*) of bookings changes mid-transaction.
Incorrect summary	An aggregate reads some values before and some after another transaction's update.	Total of all balances is wrong during a transfer.

16.3 Serializability

Type	Definition	How to test
Conflict serializability	A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping **non-conflicting** operations.	**Precedence (serialization) graph**: nodes = transactions; edge Ti → Tj if a conflicting operation of Ti precedes Tj. **No cycle ⇔ serializable.**
View serializability	View-equivalent to a serial schedule: same initial reads, same read-from relationships, same final writes. Weaker (more schedules) but NP-complete to test.	Check view-equivalence with each serial order

KEY POINT

Conflicting operations: belong to **different** transactions, access the **same item**, and **at least one is a WRITE** (R-W, W-R, W-W). Read-read never conflicts. Every conflict-serializable schedule is view-serializable, but not vice versa.

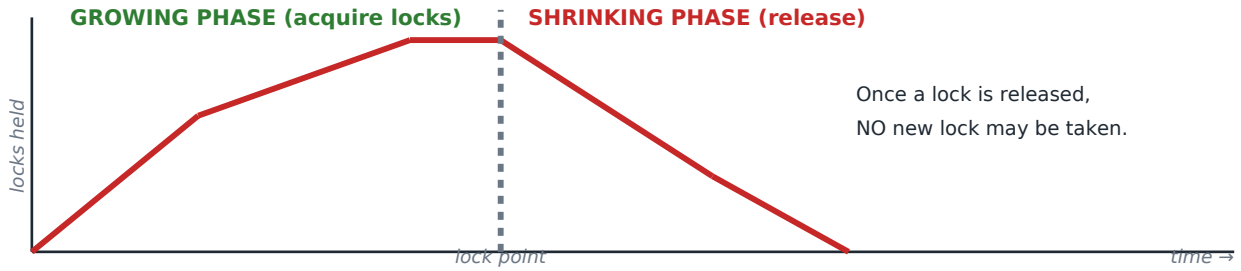
16.4 Lock-Based Protocols

Lock	Also called	Allows	Used for
Shared lock (S)	Read lock	Many transactions can hold S on the same item	Reading
Exclusive lock (X)	Write lock	Only ONE transaction; blocks all others	Writing

Held \ Requested	Shared (S)	Exclusive (X)
Shared (S)	✓ **Granted**	✗ **Wait**
Exclusive (X)	✗ **Wait**	✗ **Wait**

DEFINITION

Two-Phase Locking (2PL): each transaction has a **growing phase** (acquire locks, none released) followed by a **shrinking phase** (release locks, none acquired). 2PL guarantees **conflict serializability**.



2PL variant	Rule	Notes
Basic 2PL	Growing then shrinking phase	Serializable but may deadlock and cascade aborts
Strict 2PL	Holds all **exclusive** locks until COMMIT / ABORT	Avoids cascading rollbacks; widely used
Rigorous 2PL	Holds **all** locks until commit	Serial order = commit order
Conservative 2PL	Acquires all locks **before** starting	Deadlock-free but needs advance knowledge

16.5 Deadlocks

DEFINITION

A **deadlock** occurs when two or more transactions wait for each other's locks forever. Example: T1 holds lock on A and wants B; T2 holds lock on B and wants A.

Approach	Technique	Explanation
Prevention	Wait-Die (non-preemptive)	Older transaction may wait for younger; a **younger requesting from older dies** (aborts and restarts with the same timestamp).
Prevention	Wound-Wait (preemptive)	**Older wounds (aborts) younger** holder; younger requesting from older waits.
Prevention	Lock ordering / conservative 2PL	Acquire all locks up front or in a global order.
Detection	Wait-for graph	Node = transaction; edge Ti → Tj if Ti waits for Tj. **A cycle = deadlock.** Check periodically.
Recovery	Victim selection & rollback	Abort one transaction (fewest changes, youngest) to break the cycle; avoid starvation.
Timeout	Lock wait timeout	Abort a transaction that waits too long (simple, used by MySQL).

16.6 Timestamp Ordering and MVCC

Method	How it works	Pros / Cons
Timestamp ordering	Each transaction gets a unique timestamp TS(T) at start. Each item records the largest read-TS and write-TS. An operation that would violate timestamp order causes the transaction to **abort and restart** with a new timestamp.	No locks, no deadlocks; but more restarts and possible starvation.
Optimistic (validation)	Read → Validate → Write phases: run without locks and check for conflicts only at commit.	Good when conflicts are rare.
MVCC (Multiversion Concurrency Control)	Keeps **multiple versions** of each row. Readers see a consistent **snapshot** as of their start time; writers create new versions. **Readers never block writers and writers never block readers.**	High concurrency (PostgreSQL, Oracle, MySQL InnoDB). Costs: storage for old versions, vacuum / purge.

16.7 Transaction Isolation Levels (SQL Standard)

Isolation level	Dirty read	Non-repeatable read	Phantom read	Notes
READ UNCOMMITTED	**Possible**	**Possible**	**Possible**	Fastest, least safe
READ COMMITTED	**Prevented**	**Possible**	**Possible**	Default in PostgreSQL, Oracle, SQL Server
REPEATABLE READ	**Prevented**	**Prevented**	**Possible***	Default in MySQL InnoDB (*mostly prevented)
SERIALIZABLE	**Prevented**	**Prevented**	**Prevented**	Safest, slowest

SQL - SAFE SEAT BOOKING (PREVENTS DOUBLE BOOKING)

```sql
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;
SELECT seat_no FROM seat WHERE trip_id = 7 AND status = 'free' FOR UPDATE;  -- row lock
UPDATE seat SET status = 'booked', passenger_id = 42 WHERE trip_id = 7 AND seat_no = 12;
COMMIT;
```

EXAM TIP

Trade-off: **higher isolation = more correctness but less concurrency**. Choose the lowest level that still prevents the anomalies your application cannot tolerate.

PART 7 OF 10

Database Storage and Optimization

7

How data is physically stored and found quickly: storage structures, indexes, hashing and query optimisation.

IN THIS PART

Topic 17 Database Storage Management
Topic 18 Indexing and Hashing
Topic 19 Query Processing and Optimization

TOPIC 17

Database Storage Management

17.1 Physical Storage Structures and the Storage Hierarchy

Level	Medium	Speed	Capacity / Cost	Volatile?
Primary	CPU registers, cache (L1-L3)	Fastest (ns)	Tiny / very costly	Yes
Primary	Main memory (RAM)	Very fast	GBs / costly	Yes
Secondary	SSD / flash	Fast (µs)	TBs / moderate	No
Secondary	Magnetic hard disk (HDD)	Slow (ms: seek + rotation)	TBs / cheap	No
Tertiary	Optical disks, magnetic tape, cloud archive	Slowest	Huge / cheapest	No

KEY POINT

A DBMS must minimise **disk I/O**, because reading from disk is roughly 100,000× slower than RAM. Most database tuning (indexes, buffering, clustering) aims to reduce block reads.

17.2 Disk Organization

Term	Meaning
Platter, track, sector	A disk has circular platters; each has concentric **tracks** divided into **sectors** (smallest hardware unit, 512 B - 4 KB).
Block (page)	Unit of transfer between disk and memory (typically 4-16 KB); the DBMS reads / writes whole blocks.
Seek time	Time to move the read/write head to the correct track (largest cost).
Rotational latency	Time for the correct sector to rotate under the head (avg = half a rotation).
Transfer time	Time to read the data once positioned.
Access time	Seek + rotational latency + transfer time.
RAID	Redundant Array of Independent Disks: RAID 0 (striping, speed), RAID 1 (mirroring), RAID 5 (striping + parity), RAID 10 (mirror + stripe) for performance and fault tolerance.

17.3 File Organization

Organization	How records are stored	Good for	Weakness
Heap (unordered) file	Appended in the order of insertion	Fast inserts; full scans; bulk loading	Slow searches (scan all blocks)
Sequential (ordered) file	Sorted by a search key	Range queries, sorted reports, binary search	Insertions are costly (keep order)
Hash file	Hash function on key decides the bucket / block	Very fast equality search (≈ 1 I/O)	Poor for range queries
Indexed (ISAM / B+ tree) file	Data + index on key(s)	Both equality and range search	Index maintenance overhead
Clustered file	Records of related tables stored together	Fast joins on the cluster key	Slows other access patterns

17.4 Pages, Records and Buffer Management

Concept	Explanation
Page (block) layout	**Slotted page:** a header + slot directory pointing to records; records grow from one end, the directory from the other. A record id (RID) = (page number, slot number).
Fixed-length records	Simple, fast offsets; may waste space.
Variable-length records	Needed for VARCHAR / TEXT; use length prefixes or offset arrays; NULL bitmap.
Spanned vs unspanned	Spanned records may cross block boundaries; unspanned must fit within one block.
Buffer pool	Area of main memory that caches disk pages. A page is **pinned** while in use; a modified page is **dirty** and written back later.
Replacement policies	LRU (least recently used), Clock, MRU (for sequential scans), FIFO - decide which page to evict when the pool is full.
Force / No-force, Steal / No-steal	Policies for when dirty pages may be written: **Steal + No-force** gives best performance and needs a log for recovery (WAL).

TOPIC 18

Indexing and Hashing

18.1 Introduction to Indexing

DEFINITION

An **index** is a separate data structure (like a book's index) that maps **search-key values to record locations**, so rows can be found without scanning the whole table. The **search key** is the column(s) indexed.

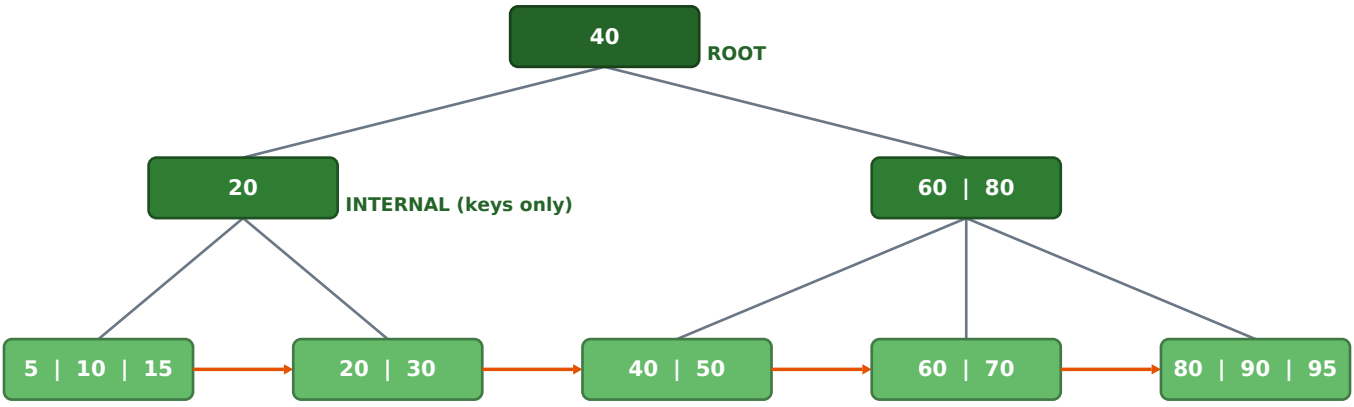
18.2 Types of Indexes

Type	Description	Notes
Primary index	Index on the **ordering key** (usually the primary key) of a sequentially ordered file	At most one per file; usually sparse
Clustering index	Index on a **non-key** ordering field; rows sorted physically by that field	One per file
Secondary index	Index on any **non-ordering** field; the data file is not sorted by it	Must be dense; many per table
Clustered index	Table rows are **physically stored** in index order (e.g. InnoDB primary key; SQL Server clustered)	Only ONE per table; fast range scans
Non-clustered index	Separate structure with pointers (RIDs / PK values) to rows	Many per table; extra lookup needed
Dense index	An index entry for **every** search-key value / record	Faster lookup, more space
Sparse index	Entries for only **some** records (one per block)	Smaller; needs a sorted file
Multilevel index	An index **on the index** (inner and outer levels) to cut I/O for large indexes	Basis of B-trees / ISAM
Composite index	Index on multiple columns; the order matters (left-most prefix rule)	INDEX(last_name, first_name)
Unique / Bitmap / Full-text / Spatial	Specialised indexes: uniqueness, low-cardinality columns, text search, GIS (R-tree)	

18.3 B-Trees and B+ Trees

DEFINITION

A **B-tree** is a balanced, multi-way search tree whose nodes equal a disk block; all leaves are at the same depth, so every search costs the same few I/Os. The **B+ tree** is the variant used by almost all DBMS indexes: **data pointers only in the leaves**, which are linked in order.



LEAVES (bottom row) hold the data pointers and are linked in order →

Feature	B-Tree	B+ Tree
Data / record pointers	In internal **and** leaf nodes	Only in **leaf** nodes
Internal nodes	Hold keys + data pointers (fewer keys per node)	Keys only → higher fan-out → shallower tree
Leaf nodes	Not linked	**Linked list** → efficient range queries and ordered scans
Search cost	Can finish at an internal node	Always reaches a leaf (predictable)
Height for 1 million keys	Typically 3-4 levels	Typically 3 levels (e.g. fan-out ~100)
Used in	Some file systems	MySQL InnoDB, PostgreSQL, Oracle, SQL Server

KEY POINT
Search, insert and delete all take **O($\log_m$ n)** time (m = fan-out). Nodes are kept at least half full by **splitting** on overflow and **merging / redistributing** on underflow, keeping the tree balanced.

18.4 Hashing

Type	How it works	Pros / Cons
Hash index	A hash function h(key) gives the bucket address; equality lookup in ~O(1)	Excellent for =; useless for ranges and ordering
Static hashing	Fixed number of buckets; **overflow chains** form when buckets fill (collisions)	Simple; degrades as data grows
Dynamic hashing	The bucket count grows and shrinks with the data	Handles growth without full reorganisation
Extendible hashing	A directory of bucket pointers indexed by the first *d* bits of the hash; the directory doubles when needed	No overflow chains; one extra directory access
Linear hashing	Splits buckets one at a time in a fixed order; no directory	Smooth, incremental growth

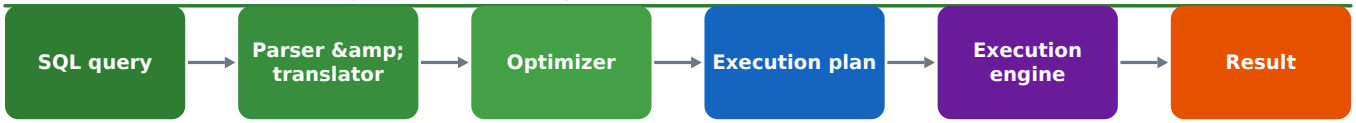
Advantages of indexing	Disadvantages of indexing
Much faster SELECT, JOIN, ORDER BY, GROUP BY	Extra storage space
Faster searches on large tables (O(log n) vs O(n))	Slower INSERT / UPDATE / DELETE (index must be maintained)
Enforces uniqueness (unique index)	Too many indexes confuse the optimizer and waste memory
Supports covering queries (index-only scans)	Need monitoring and rebuilding when fragmented

SQL - MANAGING INDEXES
```
CREATE INDEX idx_student_name ON student(name);
CREATE UNIQUE INDEX idx_student_email ON student(email);
CREATE INDEX idx_trip_route ON trip(origin, destination, depart_date);  -- composite
DROP INDEX idx_student_name ON student;
EXPLAIN SELECT * FROM student WHERE name = 'Mary';   -- check the index is used
```

TOPIC 19

Query Processing and Optimization

19.1 Steps of Query Processing



Step	What happens
1. Parsing	Checks **syntax**, verifies tables / columns exist in the catalog and the user has privileges (semantic check).
2. Translation	Converts SQL into an internal form - a relational algebra expression / query tree.
3. Optimization	Generates many equivalent plans and chooses the **cheapest** using statistics (row counts, value distribution, index info).
4. Execution plan	The chosen plan: access methods (index / scan), join order, join algorithms.
5. Execution	The engine runs the plan, using pipelining or materialisation, and returns the result.

19.2 Query Optimization

	Rule-based (heuristic) optimization	Cost-based optimization
Idea	Apply fixed transformation rules	Estimate the cost of alternative plans and pick the minimum
Examples	**Push selections down** (σ early); push projections down; combine σ and × into joins; reorder joins smallest-first	Compare I/O + CPU cost of index scan vs full scan, join order and algorithm
Needs statistics?	No	Yes (ANALYZE / UPDATE STATISTICS)
Used today	First phase	Main method in modern DBMS

19.3 Join Algorithms

Algorithm	Idea	Best when
Nested-loop join	For each row of the outer table, scan the entire inner table	Tiny tables; fallback for any condition
Block nested-loop join	Reads the outer table in blocks to reduce inner scans	No indexes; medium tables
Index nested-loop join	For each outer row, **probe an index** on the inner table	Small outer, indexed inner join column
Sort-merge join	Sort both inputs on the join key, then merge	Inputs already sorted or large; range / equi-join
Hash join	Build a hash table on the smaller table, probe it with the larger	Large unsorted tables with equality conditions

19.4 Table Scans vs Index Scans

	Full table scan	Index scan / seek
How	Reads every block of the table	Navigates the B+ tree to matching entries
Best when	Small table, or the query returns a **large fraction** of rows (>~10-20%), no usable index	Highly selective predicates (few rows), range queries, ORDER BY on the index
Index-only (covering) scan	-	All needed columns are inside the index → no table access

19.5 Query Performance Tuning

SQL - READ THE EXECUTION PLAN

```sql
EXPLAIN SELECT s.name, c.title
FROM student s JOIN enrol e ON s.id = e.student_id
               JOIN course c ON e.course_code = c.code
WHERE s.dept = 'Computing';
-- look for: type = ALL (full scan), key = NULL (no index), large "rows", "Using filesort"
```

- Index columns used in WHERE / JOIN / ORDER BY; keep statistics up to date.
- Avoid SELECT *, functions on indexed columns and leading-wildcard LIKE.
- Rewrite correlated subqueries as joins; limit result sets; batch writes.
- Monitor slow-query logs; partition very large tables; cache hot data.

PART 8 OF 10

Database Security and Recovery

8

Protecting data from misuse and loss: authentication, access control, encryption, SQL injection, backups and crash recovery.

IN THIS PART

Topic 20 Database Security
Topic 21 Backup and Recovery

TOPIC 20

Database Security

20.1 Security Fundamentals - The CIA Triad

Goal	Meaning	Controls
Confidentiality	Only authorised people can see the data	Authentication, access control, encryption
Integrity	Data is accurate and cannot be changed improperly	Constraints, privileges, audit, checksums
Availability	Data and services are accessible when needed	Backups, replication, failover, DoS protection

20.2 Authentication, Authorization and Accounts

Concept	Meaning	Examples
Authentication	**Who are you?** Verifying identity	Password, certificate, biometrics, multi-factor (MFA), OS / LDAP / Kerberos single sign-on
Authorization	**What may you do?** Granting rights after identity is proven	Privileges, roles, row-level security
User accounts	Named database logins, each with its own credentials and privileges	Principle of **least privilege**: give the minimum rights needed; remove default / unused accounts
Accounting / Auditing	Recording who did what and when	Audit logs, login history

20.3 Privileges, Roles and Role-Based Access Control (RBAC)

Privilege type	Examples
System (account-level) privileges	CREATE USER, CREATE TABLE, BACKUP, SHUTDOWN
Object privileges	SELECT, INSERT, UPDATE, DELETE, EXECUTE on specific tables / views / procedures
Column-level privileges	GRANT UPDATE (phone) ON student - only certain columns

DEFINITION

RBAC: privileges are assigned to **roles** (teacher, clerk, auditor) and users are assigned to roles. This simplifies administration: change a role once and every member is updated.

SQL - GRANT AND REVOKE

```sql
CREATE USER 'clerk1'@'localhost' IDENTIFIED BY 'Str0ng#Pass';
CREATE ROLE registrar;
GRANT SELECT, INSERT, UPDATE ON school.student TO registrar;
GRANT registrar TO 'clerk1'@'localhost';
GRANT SELECT ON school.grades TO 'teacher1'@'localhost' WITH GRANT OPTION;
REVOKE INSERT ON school.student FROM registrar;
REVOKE ALL PRIVILEGES ON *.* FROM 'clerk1'@'localhost';
```

GOOD TO KNOW

Views for security: expose only safe columns (e.g. hide salary) and grant access to the view rather than the base table. **Row-level security** restricts which rows a user can see.

20.4 Data Encryption

Type	Protects	Notes
Encryption at rest	Data stored on disk / backups	Transparent Data Encryption (TDE), column encryption, AES-256
Encryption in transit	Data on the network	TLS / SSL connections between app and database
Hashing + salting	Passwords	**Never store plain-text passwords.** Use bcrypt / Argon2 / PBKDF2 with a unique salt (one-way)
Masking / tokenisation	Sensitive fields (card, NRC)	Show only partial values to non-privileged users
Key management	Encryption keys	Store keys separately from data; rotate regularly

20.5 SQL Injection and Its Prevention

WATCH OUT

SQL injection is an attack where malicious SQL is inserted into an input field so that the application runs it. It can bypass login, leak, change or delete data.

PYTHON - VULNERABLE VS SAFE

```python
# VULNERABLE - user input concatenated into SQL
query = "SELECT * FROM users WHERE name='" + name + "' AND pass='" + pw + "'"
# attacker types   name:  admin' --      -> password check is commented out
# attacker types   name:  x'; DROP TABLE users; --

# SAFE - parameterised (prepared) statement
cursor.execute("SELECT * FROM users WHERE name = %s AND pass_hash = %s", (name, pw_hash))
```

Prevention	Detail
Parameterised queries / prepared statements	Input is treated as **data**, never as SQL code - the #1 defence
Stored procedures (with parameters)	Fixed SQL logic; no dynamic concatenation
Input validation and whitelisting	Check type, length, format on both client and server
Least-privilege database accounts	The web app must not connect as root / admin
ORM frameworks	Generate parameterised SQL automatically
Hide error details, use a WAF, keep software patched	Do not reveal table names or SQL errors to users

20.6 Auditing, Privacy and Security Policies

Area	Description
Database auditing	Logging logins, privilege changes, DDL and access to sensitive tables (audit trail) for accountability, forensics and compliance. Triggers or built-in audit features can be used.
Data privacy	Protecting personal data and respecting consent, purpose limitation, retention and the right to access / erasure. Laws and standards: GDPR, Zambia's Data Protection Act, PCI-DSS (cards), HIPAA (health).
Security policies	Written rules: password and MFA policy, who may access what, patching schedule, backup rules, incident response plan, data classification, staff training.
Other threats	Privilege abuse, weak passwords, unpatched software, insider threats, backup theft, denial-of-service, malware.

TOPIC 21

Backup and Recovery

21.1 Importance and Types of Backups

DEFINITION

A **backup** is a copy of the database stored separately so it can be restored after data loss from hardware failure, human error, attack or disaster.

Backup type	What is copied	Speed / size	Restore
Full backup	The **entire** database	Slowest, largest	Simplest - one backup
Incremental	Only data changed since the **last backup of any type**	Fastest, smallest	Slowest - need last full + **all** incrementals
Differential	Data changed since the **last full** backup	Medium (grows each day)	Medium - last full + **latest** differential
Transaction-log backup	Log records since the last log backup	Very small, frequent	Enables **point-in-time recovery**
Hot / Cold / Warm	Hot: while online; Cold: database shut down; Warm: online but read-only	-	-
Logical vs Physical	Logical: SQL export (mysqldump); Physical: raw data files	-	-

COMMAND LINE - BACKUPS

```
mysqldump -u root -p school > school_full.sql        # logical backup
mysql -u root -p school < school_full.sql             # restore
pg_dump -U postgres school > school.sql               # PostgreSQL
sqlite3 school.db ".backup backup.db"                 # SQLite
```

EXAM TIP

3-2-1 rule: keep **3** copies of your data, on **2** different media, with **1** copy off-site (or in the cloud). Always **test restores**.

21.2 Failure Types

Failure	Cause	Effect / Recovery
Transaction failure	Logical error (bad input, divide by zero) or system error (deadlock)	Roll back the transaction (UNDO)
System crash	Power loss, OS or DBMS crash; main memory lost	Memory lost, disk intact → crash recovery with the log (REDO / UNDO)
Media (disk) failure	Disk head crash, corruption, fire	Restore from backup + replay logs; RAID / replicas
Human / security failure	Accidental DROP / DELETE, malware, ransomware	Point-in-time recovery from backups
Disaster	Flood, fire, theft, data-centre outage	Disaster recovery site, off-site backups

21.3 Transaction Log, Checkpoints and Log-Based Recovery

DEFINITION

The **transaction log** is a sequential file recording every change: <T, start>, <T, X, old value, new value>, <T, commit>, <T, abort>. A **checkpoint** forces buffered changes to disk and records the active transactions, limiting how much of the log must be examined after a crash.

Technique	How it works	Recovery action
Deferred database modification	Writes are logged first and applied to the database **only at commit**	Only **REDO** committed transactions; ignore uncommitted ones
Immediate database modification	Updates may be written to the DB **before commit** (after logging)	**UNDO** uncommitted transactions; **REDO** committed ones
Write-Ahead Logging (WAL)	The **log record must reach stable storage BEFORE** the changed data page is written; and all log records before COMMIT returns	Guarantees atomicity (undo) and durability (redo)
ARIES algorithm	WAL + checkpoints + repeating history. Three passes after a crash:	**1. Analysis** (find dirty pages and active txns) → **2. Redo** (repeat history) → **3. Undo** (roll back losers)
Shadow paging	Keeps a **current** and a **shadow** page table; updates go to new pages; commit = atomic switch of the pointer	No log needed; discard the current table to abort. Fragmentation and overhead

KEY POINT

Crash recovery summary: committed transactions are **redone** (durability); uncommitted transactions are **undone** (atomicity). Checkpoints shorten the log scan.

21.4 Disaster Recovery, RPO and RTO

Term	Meaning	Example
Disaster Recovery (DR)	Plans, sites and procedures to restore IT services after a major disaster	Secondary data centre, cloud failover, replication
RPO - Recovery Point Objective	Maximum **acceptable data loss**, measured in time (how old may the restored data be?)	RPO = 15 min → back up / replicate at least every 15 min
RTO - Recovery Time Objective	Maximum **acceptable downtime** (how fast must service return?)	RTO = 1 hour → recovery must finish in 1 hour
High availability	Redundant servers with automatic failover; replication	Primary + standby (hot standby)
DR testing	Regularly rehearse restores and failovers	Quarterly restore drills

EXAM TIP

A **small RPO** needs frequent backups / continuous replication; a **small RTO** needs standby systems and automation. Both cost more as they approach zero.

PART 9 OF 10

Advanced Database Systems

Beyond a single server: distributed databases, NoSQL, data warehousing and OLAP, and modern database technologies.

9

IN THIS PART

Topic 22	Distributed Databases
Topic 23	NoSQL Databases
Topic 24	Data Warehousing and OLAP
Topic 25	Modern Database Technologies

TOPIC 22

Distributed Databases

22.1 Introduction

DEFINITION

A **distributed database (DDB)** is a collection of logically related databases spread across multiple sites (computers) connected by a network, managed by a **distributed DBMS** that makes them appear as one database to the user (**distribution transparency**).

	Centralized database	Distributed database
Location of data	One site	Many sites / regions
Failure	Single point of failure	Survives site failures (replication)
Performance	Bottleneck under load; remote users slow	Data close to users; parallel processing
Scalability	Scale up (bigger machine)	Scale out (more machines)
Complexity / cost	Simple	Complex design, security and consistency challenges

22.2 Fragmentation, Replication and Allocation

Technique	Description	Example
Horizontal fragmentation	Split a table by **rows** (using a predicate); rebuilt with UNION	STUDENT rows for Lusaka campus at site 1, Ndola campus at site 2
Vertical fragmentation	Split a table by **columns**, each fragment keeping the primary key; rebuilt with JOIN	EMPLOYEE(ID, Name) at HR; EMPLOYEE(ID, Salary) at Finance
Mixed (hybrid) fragmentation	Both horizontal and vertical	-
Replication	Storing **copies** of data on several sites. Full, partial or none. Synchronous (strong consistency, slower) or asynchronous (fast, may lag)	Master-slave; multi-master
Data allocation	Deciding **which fragment / replica goes to which site** (non-redundant, partially or fully replicated)	Place data where it is used most

KEY POINT

Fragmentation rules: completeness (no data lost), reconstruction (can rebuild the original), disjointness (no overlap in horizontal fragments).

22.3 Distributed Transactions and Two-Phase Commit (2PC)

DEFINITION

A distributed transaction touches data at several sites and must still be **atomic**: all sites commit or all abort. **Two-Phase Commit** achieves this with a **coordinator** and **participants**.

Phase	Coordinator	Participants
Phase 1 - Prepare (Voting)	Sends **PREPARE** to all participants	Write to log, then reply **YES (ready)** or **NO (abort)**
Phase 2 - Commit / Abort (Decision)	If **all YES** → logs and sends **COMMIT**; if any NO or timeout → sends **ABORT**	Apply the decision, release locks, send **ACK**

WATCH OUT

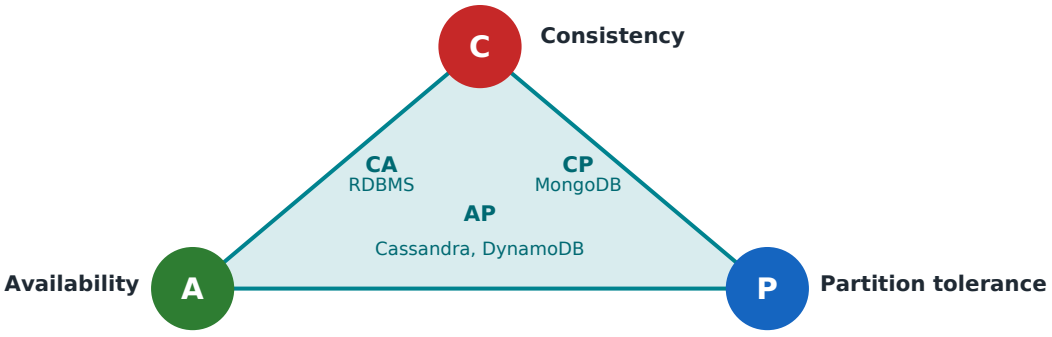
Weakness - blocking: if the coordinator crashes after participants voted YES, they must wait with locks held. **Three-Phase Commit (3PC)** adds a pre-commit phase to reduce blocking; **Paxos / Raft** consensus and **Saga** patterns are used in modern systems.

Distributed concurrency control & query processing	
Distributed locking	Centralized lock manager, primary copy, or distributed lock manager; global deadlock detection.
Timestamp / MVCC	Globally unique timestamps (logical clocks, hybrid clocks).
Distributed query processing	Minimise **data transferred over the network**: use **semi-joins**, send the smaller relation, push selections to remote sites, choose join sites by cost.

22.4 CAP Theorem and Consistency Models

DEFINITION

CAP theorem (Brewer): a distributed data store can guarantee at most **two** of three: **C**onsistency (every read sees the latest write), **A**vailability (every request gets a response), **P**artition tolerance (system works despite network splits). Since partitions are unavoidable, the real choice is between **C and A during a partition**.



Model	Guarantee	Used by
Strong (linearizable) consistency	Every read returns the most recent write; behaves like one copy	Banking, CP systems (Spanner, HBase)
Eventual consistency	If no new updates, all replicas **eventually** converge; reads may be stale for a while	DynamoDB, Cassandra, DNS
Causal consistency	Causally related operations are seen in order by everyone	Collaboration tools
Read-your-writes / session	A user always sees their own updates	Web sessions

Model	Guarantee	Used by
ACID vs BASE	**ACID** (strict, traditional) vs **BASE**: Basically Available, Soft state, Eventually consistent	RDBMS vs many NoSQL

TOPIC 23

NoSQL Databases

23.1 Introduction and SQL vs NoSQL

DEFINITION

NoSQL ("Not Only SQL") databases store data in non-tabular models, with flexible schemas and **horizontal scalability**, built for big data, high velocity and varied data types.

Feature	SQL (Relational)	NoSQL
Data model	Tables with rows and columns	Key-value, document, column-family, graph
Schema	Fixed, defined in advance (schema-on-write)	Flexible / dynamic (schema-on-read)
Scaling	Mainly vertical (bigger server)	Horizontal (add nodes, sharding)
Transactions	Full ACID	Often BASE; some support ACID per document
Relationships	Joins and foreign keys	Embedding or references; limited joins
Query language	Standard SQL	Database-specific APIs / query languages
Best for	Structured data, complex queries, strong integrity	Massive scale, unstructured / changing data, low latency

23.2 Types of NoSQL Databases

Type	Model	Example product	Typical use cases
Key-value	Simple key → value pairs (like a dictionary / hash map)	**Redis**, DynamoDB, Memcached	Caching, sessions, shopping carts, leaderboards, rate limiting
Document	JSON / BSON documents with nested fields in collections	**MongoDB**, CouchDB, Firestore	Content management, catalogs, user profiles, mobile backends
Column-family (wide-column)	Rows with dynamic column families, partitioned across nodes	**Cassandra**, HBase, Bigtable	Time-series, IoT, logging, massive write workloads
Graph	Nodes (entities) and edges (relationships) with properties	**Neo4j**, Amazon Neptune, ArangoDB	Social networks, recommendations, fraud detection, knowledge graphs

NOSQL QUERY EXAMPLES

```
// MongoDB (document)
db.students.insertOne({ name: "Mary", age: 20, courses: ["DBMS", "IoT"] })
db.students.find({ age: { $gt: 18 } })
db.students.updateOne({ name: "Mary" }, { $set: { age: 21 } })

// Redis (key-value)
SET session:42 "logged_in"   EX 3600
GET session:42
INCR page:views

// Neo4j (graph, Cypher)
MATCH (a:Person {name:"Mary"})-[:FRIEND_OF]->(f) RETURN f.name
```

23.3 Advantages, Limitations and Applications

Advantages	Limitations
Horizontal scaling and high availability	Weaker consistency guarantees (eventual)
Flexible schema - fast development	No standard query language; less tooling
High performance for specific access patterns	Complex joins / multi-document transactions are limited
Handles unstructured and semi-structured data	Data duplication from denormalized designs
Cloud-native and distributed by design	Skills and ecosystem still maturing

GOOD TO KNOW

Applications: social media feeds, real-time analytics, IoT sensor streams, gaming, recommendation engines, e-commerce catalogs, mobile apps, log management, fraud detection.

TOPIC 24

Data Warehousing and OLAP

24.1 Introduction and Characteristics

DEFINITION

A **data warehouse** is a central repository that integrates data from many operational sources, optimised for **analysis and reporting** rather than daily transactions (Bill Inmon).

Characteristic	Meaning
Subject-oriented	Organised around business subjects (sales, customers) not applications
Integrated	Data from many sources made consistent (formats, codes, units)
Time-variant	Stores historical data over years; every record is time-stamped
Non-volatile	Data is loaded and read, not updated or deleted in place

24.2 OLTP versus OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Purpose	Run day-to-day operations	Analysis, decision support, reporting
Operations	Many short INSERT / UPDATE / DELETE / simple SELECT	Complex, long-running aggregate queries
Data	Current, detailed, normalized (3NF)	Historical, summarized, denormalized (star schema)
Users	Clerks, customers, many concurrent users	Analysts, managers, executives
Size	MBs - GBs	GBs - PBs
Optimised for	Write speed, integrity, concurrency	Read speed, scanning, aggregation
Example	Bank ATM, ticket booking, mobile money	Monthly sales trend by region

24.3 ETL and ELT



	ETL	ELT
Order	Extract → Transform → Load	Extract → Load → Transform

	ETL	ELT
Where transformation happens	In a separate staging / ETL engine	Inside the target warehouse (cloud power)
Best for	Traditional warehouses, strict cleansing before loading	Cloud warehouses and data lakes (BigQuery, Snowflake)

24.4 Dimensional Modelling: Facts, Dimensions and Schemas

Element	Description	Example
Fact table	Central table of **measurable events** (numeric measures) plus foreign keys to dimensions. Very large. Types: transaction, periodic snapshot, accumulating snapshot, factless.	FACT_SALES(date_key, product_key, store_key, quantity, amount)
Dimension table	**Descriptive context** (who, what, where, when). Wide, smaller, denormalized, with a surrogate key.	DIM_PRODUCT, DIM_DATE, DIM_STORE, DIM_CUSTOMER
Measure	Numeric value that can be aggregated (additive, semi-additive, non-additive).	Quantity, amount
Grain	The level of detail of one fact row - decide it first!	One row per product per sale line



Schema	Structure	Pros	Cons
Star schema	One fact table surrounded by **denormalized** dimension tables	Simple, fast queries, fewer joins	Some redundancy in dimensions
Snowflake schema	Dimensions **normalized** into sub-dimensions (Product → Category → Department)	Less redundancy, saves space	More joins, more complex, slower
Galaxy (fact constellation)	**Multiple fact tables** sharing conformed dimensions	Models several business processes	Most complex to design

24.5 Data Marts, Slowly Changing Dimensions, OLAP Operations

Concept	Explanation
Data mart	A smaller, focused subset of a warehouse for one department (sales, finance). Approaches: top-down (Inmon, mart from warehouse) or bottom-up (Kimball, warehouse from marts).
Data cube	A multidimensional array of measures along dimensions (e.g. Product × Region × Time) enabling fast aggregation.
Business Intelligence (BI)	Tools and processes (dashboards, reports, KPIs - Power BI, Tableau) that turn warehouse data into insight.
Warehouse architecture	Sources → staging area (ETL) → central warehouse (+ data marts, metadata) → OLAP server / BI tools → users.

SCD type	How a changed attribute is handled	History kept?
Type 0	Never changes (retain original)	n/a

SCD type	How a changed attribute is handled	History kept?
Type 1	**Overwrite** the old value	No
Type 2	**Add a new row** with effective-from / to dates and current flag	Full history
Type 3	Add a **new column** for the previous value	Limited (one prior value)
Type 4 / 6	History table (4); hybrid of 1 + 2 + 3 (6)	Yes

OLAP operation	Meaning	Example
Roll-up (drill-up)	Aggregate to a **higher** level or reduce dimensions	City → Province → Country
Drill-down	Go to **more detail**	Year → Quarter → Month → Day
Slice	Fix **one** dimension value to get a sub-cube	Only year = 2026
Dice	Select a sub-cube using **several** dimension conditions	2026, Region IN (Copperbelt, Lusaka), Product = Phones
Pivot (rotate)	Rotate the axes for a different view	Swap rows and columns of a report

SQL - OLAP STYLE QUERY

```sql
SELECT d.year, p.category, SUM(f.amount) AS revenue
FROM fact_sales f
JOIN dim_date d    ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY ROLLUP (d.year, p.category);     -- subtotals per year + grand total
```

TOPIC 25

Modern Database Technologies

Technology	What it is	Examples / use
Cloud databases	Databases running on cloud infrastructure; pay-as-you-go, elastic, managed backups	Amazon RDS / Aurora, Azure SQL, Google Cloud SQL
DBaaS (Database as a Service)	Provider manages installation, patching, scaling, backup and HA; you just use the database	MongoDB Atlas, Supabase, Firebase, DynamoDB
NewSQL	Relational model + SQL + **ACID** with NoSQL-style horizontal scalability	Google Spanner, CockroachDB, TiDB, VoltDB
In-memory databases	Keep all data in **RAM** for microsecond access; snapshots / logs for durability	Redis, SAP HANA, Memcached; real-time bidding, caching
Time-series databases	Optimised for time-stamped data: heavy ingest, compression, downsampling, time-window queries	InfluxDB, TimescaleDB, Prometheus - IoT, monitoring, finance
Graph databases	Store nodes and relationships natively; fast multi-hop traversals	Neo4j - social networks, fraud rings, recommendations
Spatial databases	Store and query geometry (points, lines, polygons) with spatial indexes (R-tree), GIS functions	PostGIS, Oracle Spatial - maps, routes, bird migration tracking
Vector databases	Store **embeddings** (numeric vectors) and find nearest neighbours (ANN) for similarity search	Pinecone, Milvus, pgvector - AI search, RAG, recommendations
Database automation	Self-tuning / self-healing databases: auto-indexing, auto-scaling, AI-driven optimisation, DevOps (infrastructure as code, CI/CD for schemas)	Autonomous Database, Aurora auto-scaling
Database scalability	**Vertical**: bigger machine. **Horizontal**: more machines (sharding, replication, partitioning)	Scale to millions of users
Sharding	Horizontal partitioning of data across **independent servers** by a shard key (range, hash, directory)	Users A-M on shard 1, N-Z on shard 2. Risks: hot spots, cross-shard joins
Database replication	Copying data to other nodes: primary-replica (read scaling, failover), multi-primary, sync / async	MySQL replication, PostgreSQL streaming replication
Big data storage systems	Distributed storage for huge, varied data: HDFS, data lakes (S3), Hadoop / Spark, lakehouse (Delta Lake)	Analytics, machine learning pipelines

EXAM TIP

Choosing a database: structured + transactions → RDBMS (PostgreSQL / MySQL); flexible documents → MongoDB; caching → Redis; relationships → Neo4j; time-stamped sensors → time-series DB; analytics → data warehouse; similarity search → vector DB; global scale with SQL → NewSQL.

PART 10 OF 10

Practical Database Development

10

Putting theory to work: DBMS software, connecting databases to programs, and building complete real-world database systems.

IN THIS PART

Topic 26 MySQL Practical Training
Topic 27 Connecting Databases to Programming Languages
Topic 28 Real-World Database Projects

TOPIC 26

MySQL Practical Training

GOOD TO KNOW
Learn by doing: install a DBMS, create a database, build tables, insert data, query it, then connect it to a program. Practise every day.

Software	Type	Key tools / what to practise
MySQL Server	Open-source RDBMS	mysql CLI, users and privileges, InnoDB engine, stored routines, replication
MySQL Workbench	GUI client	Visual ER modelling (EER), forward / reverse engineering, query editor, performance reports
PostgreSQL	Open-source object-relational	psql, pgAdmin, JSONB, window functions, PostGIS extension
Microsoft SQL Server	Commercial RDBMS	SSMS, T-SQL, SQL Agent jobs, Always On availability
SQLite	Embedded, serverless, file-based	sqlite3 CLI, one-file database; perfect for learning and mobile / small apps
Oracle Database	Commercial enterprise RDBMS	SQL*Plus / SQL Developer, PL/SQL, partitioning; Oracle XE is free
MongoDB	NoSQL document	mongosh, Compass, collections, aggregation pipeline, indexes

SQL - PRACTICE SESSION

```sql
-- A first MySQL session
mysql -u root -p
CREATE DATABASE bus_system;
USE bus_system;
SHOW DATABASES;  SHOW TABLES;  DESCRIBE route;

CREATE TABLE route (
  route_id    INT PRIMARY KEY AUTO_INCREMENT,
  origin      VARCHAR(40) NOT NULL,
  destination VARCHAR(40) NOT NULL,
  fare        DECIMAL(8,2) CHECK (fare > 0)
);
INSERT INTO route (origin, destination, fare) VALUES ('Kitwe','Lusaka',250.00),('Ndola','Chinsali',300.00);
SELECT * FROM route WHERE fare < 280;
```

Daily practice plan	
Week 1	Install MySQL + Workbench; CREATE, INSERT, SELECT, UPDATE, DELETE; constraints
Week 2	Design an ERD, convert to tables, add keys; WHERE, ORDER BY, GROUP BY, HAVING
Week 3	All joins, subqueries, views; normalise a messy table to 3NF
Week 4	Transactions, indexes + EXPLAIN, procedures, triggers, backup / restore; connect from Python

TOPIC 27

Connecting Databases to Programming Languages

Language + DB	Library / driver	Notes
Python + MySQL	mysql-connector-python, PyMySQL	pip install mysql-connector-python
Python + SQLite	sqlite3 (built in)	No server needed
Python + PostgreSQL	psycopg2 / psycopg 3	pip install psycopg2-binary
PHP + MySQL	PDO or MySQLi	Use PDO prepared statements
Java + MySQL	JDBC (mysql-connector-j)	DriverManager / DataSource
C# + SQL Server	ADO.NET (SqlConnection), Entity Framework	System.Data.SqlClient / Microsoft.Data.SqlClient
Any language + database	REST / GraphQL **APIs**, ODBC	API sits between the app and the database

PYTHON + SQLITE

```python
import sqlite3                      # Python + SQLite (works everywhere)
conn = sqlite3.connect("bus.db")
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS seat (trip INT, no INT, status TEXT, PRIMARY KEY(trip, no))")
cur.execute("INSERT OR IGNORE INTO seat VALUES (?, ?, ?)", (7, 12, "free"))
conn.commit()                       # make it permanent
for row in cur.execute("SELECT * FROM seat WHERE status = ?", ("free",)):
    print(row)
conn.close()
```

PYTHON + MYSQL WITH A TRANSACTION

```python
import mysql.connector              # Python + MySQL
conn = mysql.connector.connect(host="localhost", user="app", password="secret", database="bus_system")
cur = conn.cursor(dictionary=True)
try:
    cur.execute("UPDATE seat SET status=%s WHERE trip=%s AND no=%s AND status='free'", ("booked", 7, 12))
    if cur.rowcount == 0:
        raise Exception("Seat already taken")      # prevents double booking
    conn.commit()
except Exception as e:
    conn.rollback(); print("Failed:", e)
finally:
    cur.close(); conn.close()
```

JAVA + JDBC

```java
// Java + JDBC
try (Connection c = DriverManager.getConnection("jdbc:mysql://localhost/school", "app", "secret");
     PreparedStatement ps = c.prepareStatement("SELECT name FROM student WHERE age > ?")) {
    ps.setInt(1, 18);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) System.out.println(rs.getString("name"));
}
```

ORM, SQLAlchemy and Connection Pooling

Concept	Explanation
ORM (Object-Relational Mapping)	Maps **tables to classes**, rows to objects and columns to attributes, so you work with objects instead of writing SQL. Examples: SQLAlchemy (Python), Hibernate (Java), Entity Framework (C#), Eloquent (PHP), Django ORM.
ORM advantages	Less boilerplate, database-independent code, automatic parameterisation (safer), migrations.
ORM drawbacks	Hidden inefficient SQL (the N+1 query problem), learning curve, less control for complex queries.
SQLAlchemy	Python toolkit with a Core (SQL expression language) and an ORM layer.
Connection pooling	Opening a connection is expensive. A **pool** keeps a set of open connections that are borrowed and returned, improving speed and limiting load on the server (HikariCP, SQLAlchemy pool, PgBouncer).

PYTHON - SQLALCHEMY ORM

```python
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, Session

Base = declarative_base()
class Student(Base):
    __tablename__ = "student"
    id = Column(Integer, primary_key=True)
    name = Column(String(50))

engine = create_engine("sqlite:///school.db", pool_size=5)  # pool for server DBs
Base.metadata.create_all(engine)
with Session(engine) as s:
    s.add(Student(name="Mary")); s.commit()
    print(s.query(Student).filter(Student.name == "Mary").all())
```

TOPIC 28

Real-World Database Projects

EXAM TIP

To become good at DBMS, **design and implement complete systems**: gather requirements → ER diagram → relational schema → normalize (3NF) → create tables with constraints → insert test data → write queries, views, procedures, triggers → add transactions, indexes, security and backup → connect an application.

No.	Practical project	Main concepts covered	Core tables to design
1	Student Management System	Tables, relationships, SQL	student, course, enrolment, lecturer, department
2	Library Management System	Primary and foreign keys	book, author, member, loan, publisher
3	Hospital Management System	ERD and normalization	patient, doctor, appointment, prescription, ward, bill
4	Banking Management System	Transactions and ACID	customer, account, transaction, branch, loan
5	Bus Ticket Booking System	Relationships and concurrency	route, bus, trip, seat, passenger, booking, payment
6	Inventory Management System	SQL queries and triggers	product, supplier, stock, purchase_order, sales (trigger updates stock)
7	Hotel Reservation System	Joins and constraints	room, room_type, guest, reservation, payment
8	School Examination System	Aggregate functions	student, subject, exam, result (AVG, MAX, RANK)
9	Online Shopping System	Transactions and security	customer, product, cart, order, order_item, payment
10	Employee Payroll System	Stored procedures	employee, department, salary_grade, payroll, deductions
11	University Registration System	Database design	student, programme, course, semester, registration, fees
12	Mobile Money System	Transaction management	wallet, user, transaction, agent, fee (atomic transfers)

No.	Practical project	Main concepts covered	Core tables to design
13	Church Management System	Relational database design	member, family, ministry, attendance, donation, event
14	Bird Migration Tracking System	Spatial and time-series data	species, bird, tracker, location_reading (lat, lon, time) - PostGIS / time-series
15	Data Warehouse for Supermarkets	ETL, OLAP and dimensional modelling	fact_sales; dim_date, dim_product, dim_store, dim_customer

Mini Case Study: Bus Ticket Booking (concurrency)

EXAMPLE

Problem: Two passengers try to book seat 12 on the same trip at the same instant. **Solution:** run the booking inside a transaction, lock the seat row (`SELECT ... FOR UPDATE`) or use a conditional update (`UPDATE ... WHERE status = 'free'`) and check the affected row count; add a UNIQUE (trip_id, seat_no) constraint on BOOKING as a final safety net.

SQL - CONSTRAINT PREVENTS DOUBLE BOOKING

```sql
CREATE TABLE booking (
  booking_id   INT PRIMARY KEY AUTO_INCREMENT,
  trip_id      INT NOT NULL,
  seat_no      INT NOT NULL,
  passenger_id INT NOT NULL,
  booked_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (trip_id, seat_no),                    -- two people can never hold the same seat
  FOREIGN KEY (trip_id)      REFERENCES trip(trip_id),
  FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id)
);
```

Recommended Study Order and Roadmap

GOOD TO KNOW
For a university Data Science student: follow the stages in order. Beginner stages build vocabulary and design skills; intermediate stages build the core database skills; advanced stages cover what makes databases reliable, fast and scalable; professional stage applies everything.

Stage	Topics	Level	Where in these notes
1	Introduction to DBMS and File Systems	Beginner	Topic 1
2	Database Architecture	Beginner	Topic 2
3	Database Models	Beginner	Topic 3
4	ER Diagrams	Beginner	Topic 4
5	Relational Database Model and Keys	Intermediate	Topic 5
6	Functional Dependencies	Intermediate	Topic 6
7	Database Normalization	Intermediate	Topic 7
8	SQL Fundamentals	Intermediate	Topics 8-10
9	SQL Joins and Advanced Queries	Intermediate	Topics 11-12
10	Relational Algebra and Calculus	Intermediate	Topics 13-14
11	Transactions and ACID Properties	Advanced	Topic 15
12	Concurrency Control	Advanced	Topic 16
13	Storage, Indexing and Hashing	Advanced	Topics 17-18
14	Query Processing and Optimization	Advanced	Topic 19
15	Database Security and Recovery	Advanced	Topics 20-21
16	Distributed Databases	Advanced	Topic 22
17	NoSQL Databases	Advanced	Topic 23
18	Data Warehousing and OLAP	Advanced	Topic 24
19	Modern Database Technologies	Advanced	Topic 25
20	Practical Database Projects	Professional	Topics 26-28

Quick Revision Sheet

Key Definitions in One Line

Term	One-line definition
DBMS	Software that creates, stores, secures and manages databases and controls access to them.
Schema / Instance	The structure of a database / the data in it at a moment in time.
Primary key	Minimal chosen attribute set that uniquely identifies each row; cannot be NULL.
Foreign key	Attribute referencing a primary key; enforces referential integrity.
Functional dependency	X → Y: the value of X determines the value of Y.
1NF / 2NF / 3NF	Atomic values / no partial dependency / no transitive dependency.
BCNF	Every determinant of a non-trivial FD is a super key.
ACID	Atomicity, Consistency, Isolation, Durability.
2PL	Growing phase (acquire locks) then shrinking phase (release locks) → serializable.
Deadlock	Cycle of transactions waiting for each other's locks (detect with a wait-for graph).
MVCC	Multiple row versions so readers do not block writers.
B+ tree	Balanced multi-way tree; keys in internal nodes, data pointers in linked leaves.
WAL	Log records are written to disk before the data pages they describe.
CAP	During a partition choose Consistency or Availability.
2PC	Prepare (vote) phase then Commit / Abort (decision) phase for distributed atomicity.
OLTP vs OLAP	Many small transactions vs complex analytical queries.
Star schema	Fact table surrounded by denormalized dimension tables.
RPO / RTO	Maximum data loss allowed / maximum downtime allowed.

SQL Cheat Sheet

```
SQL - CLAUSE ORDER AND MAIN COMMANDS
SELECT col1, AGG(col2) FROM t1
JOIN t2 ON t1.k = t2.k
WHERE condition                -- row filter
GROUP BY col1
HAVING AGG(col2) > n           -- group filter
ORDER BY col1 DESC
LIMIT n OFFSET m;

INSERT INTO t (a,b) VALUES (1,'x');      UPDATE t SET a=2 WHERE id=1;      DELETE FROM t WHERE id=1;
CREATE TABLE / ALTER TABLE / DROP TABLE / TRUNCATE TABLE
GRANT priv ON obj TO user;   REVOKE priv ON obj FROM user;
START TRANSACTION; ... COMMIT;  or  ROLLBACK;
```

Common Exam Comparisons

Compare	Key difference
DELETE / TRUNCATE / DROP	Remove some rows (DML, rollback-able) / remove all rows fast (DDL) / remove the whole table.
WHERE / HAVING	Filters rows before grouping / filters groups after aggregation.
UNION / UNION ALL	Removes duplicates / keeps duplicates.
RANK / DENSE_RANK / ROW_NUMBER	Gaps after ties / no gaps / unique numbers.
Clustered / non-clustered index	Rows stored in index order (one per table) / separate structure with pointers (many).
Dense / sparse index	Entry for every record / entry for some records (one per block).
B-tree / B+ tree	Data in all nodes / data only in linked leaves.
Shared / exclusive lock	Read - many holders / write - one holder.
Wait-die / wound-wait	Younger dies when requesting from older / older aborts younger holder.
Full / incremental / differential backup	Everything / since last backup / since last full backup.
ETL / ELT	Transform before loading / transform after loading in the warehouse.
Star / snowflake schema	Denormalized dimensions / normalized dimensions.
SQL / NoSQL	Fixed schema, ACID, joins, vertical scale / flexible schema, BASE, horizontal scale.
Relational algebra / calculus	Procedural (how) / declarative (what); equal expressive power.

KEY POINT

Final study tips: (1) draw ERDs by hand until it is natural; (2) normalise at least ten tables; (3) write SQL daily and read EXPLAIN plans; (4) trace schedules and draw precedence graphs; (5) build one full project (e.g. bus booking) end to end; (6) teach a topic to a friend - if you can explain it, you know it.